



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Dipartimento di Ingegneria "Enzo Ferrari"

Corso di Laurea Triennale in Ingegneria Informatica (L-08)

Tecniche di Quantizzazione Avanzate per Modelli Linguistici di Grandi Dimensioni: Analisi Teorica e Valutazione Sperimentale su Supercomputer Leonardo

Relatore:

Chiar.mo Prof. Lorenzo Baraldi

Candidata:

Ester Vecchio

Matricola 187790

ANNO ACCADEMICO 2025/2026

Una vita senza ricerca non è degna di essere vissuta.

— *Apologia di Socrate*, Platone

Sommario

La recente diffusione dei Modelli Linguistici di Grandi Dimensioni (*Large Language Models*, LLM) ha rivoluzionato il panorama dell'Intelligenza Artificiale, dimostrando straordinarie capacità di comprensione, ragionamento e generazione del linguaggio naturale. Tuttavia, la ripida scalata del numero di parametri che supera regolarmente decine o centinaia di miliardi impone requisiti computazionali e di memoria estremamente gravosi, creando una forte barriera all'adozione sostenibile ed efficiente di queste tecnologie. In fase di inferenza autoregressiva, la velocità di generazione è pesantemente limitata dal trasferimento continuo dei pesi dalla memoria principale agli acceleratori hardware, configurando il noto collo di bottiglia della larghezza di banda (*Memory Wall*).

In questo contesto, le tecniche di compressione numerica e in particolare la **Quantizzazione** rappresentano una delle risposte metodologiche più efficaci e promettenti. Riducendo la precisione dei parametri da formati ad alta precisione (come FP16 o BF16 a 16 bit) a formati a bassa precisione (quali INT8, INT4 e rappresentazioni ternarie sub-4-bit), è possibile abbattere il consumo di memoria VRAM fino a un fattore superiore a $3\times$, accelerare il throughput di inferenza e ridurre drasticamente il consumo energetico per singolo token generato.

Il presente elaborato di tesi si propone di analizzare prima i fondamenti teorico-matematici, poi l'applicazione pratica delle principali tecniche di quantizzazione. Lo studio approfondisce le metodologie di **Post-Training Quantization (PTQ)** analizzando la quantizzazione simmetrica Min-Max e algoritmi di secondo ordine basati su inversione dell'Hessiana quali GPTQ, SmoothQuant e AWQ e di **Quantization-Aware Training (QAT)**, esplorando l'impiego della *fake quantization* unita a tecniche di fine-tuning a basso rango (*LoRA*).

La validazione sperimentale è stata sviluppata ed eseguita sulla partizione Booster del

supercomputer Tier-0 Leonardo, ospitato presso il data center **CINECA** di Bologna e dotato di GPU NVIDIA Custom A100 SXM4 da 64GB. L'accesso a tale infrastruttura di calcolo ad alte prestazioni, concesso nell'ambito delle attività di tesi coordinate dal relatore Prof. Lorenzo Baraldi, ha consentito di quantizzare, compilare tramite `torch.compile` e valutare modelli di riferimento all'avanguardia quali **Meta-Llama-3.1-8B** e **Google Gemma-2-9B**. L'elaborato documenta l'intera architettura sperimentale predisposta per la misurazione offline di Time-To-First-Token (TTFT), velocità di generazione in token al secondo, picco di memoria grafica allocata e perplexity, delineando una guida metodologica e prospettive future verso paradigmi a bassissimo consumo energetico (*Green AI*).

Indice

Sommario	II
1 Introduzione e Fondamenti di Machine Learning	1
1.1 Motivazione e Contesto della Ricerca	1
1.2 Obiettivi dell'Elaborato e Contributo Sperimentale	2
1.3 Struttura dell'Elaborato	3
1.4 Il Paradigma dell'Apprendimento Automatico	4
1.5 Dalle Reti Neurali al Deep Learning	5
1.6 Acceleratori Hardware e Architetture di Calcolo	6
1.6.1 Gerarchia di Memoria e Tensor Cores	6
1.7 Valutazione delle Prestazioni: Il Modello Roofline e i Costi Computazionali .	7
2 I Modelli Linguistici di Grandi Dimensioni (LLM)	9
2.1 Cos'è un Modello Linguistico: Definizione, Classificazione e Relazione con la Quantizzazione	9
2.2 L'Architettura Transformer e i Meccanismi di Attenzione	11
2.2.1 Feed-Forward Networks e Varianti Moderne	14
2.3 Tassonomia dei Modelli: Dai PLM agli LLM Generativi	15
2.4 La Dinamica di Inferenza: Prefill, Decode e la KV Cache	15
2.5 I Limiti Computazionali e il Problema della Memoria	16
3 Teoria e Tecniche di Quantizzazione	18
3.1 Rappresentazione Numerica nei Sistemi di Calcolo	18
3.2 Formalizzazione Matematica della Quantizzazione	19

3.2.1	Quantizzazione Uniforme Affine (Asimmetrica)	19
3.2.2	Quantizzazione di Scala Simmetrica	20
3.2.3	Granularità della Quantizzazione	21
3.2.4	Metodologie di Calibrazione	23
3.3	Post-Training Quantization (PTQ)	24
3.3.1	Weight-Only vs Weight-Activation Quantization	24
3.3.2	Il Problema degli Outlier di Attivazione e SmoothQuant	25
3.3.3	Algoritmi PTQ Avanzati di Secondo Ordine: GPTQ e AWQ	26
3.4	Quantization-Aware Training (QAT)	27
3.4.1	Il Problema del Gradiente e lo Straight-Through Estimator (STE) . .	27
3.4.2	Simulazione di Fake Quantization	28
3.4.3	QAT Efficace con Low-Rank Adaptation (LoRA / QLoRA)	28
3.5	Frontiere Estreme: Modelli a 1-Bit e Architettura BitNet b1.58	29
4	Metodologia Sperimentale sul Supercomputer Leonardo	31
4.1	L’Infrastruttura di High Performance Computing: Leonardo	31
4.1.1	Caratteristiche Hardware dei Nodi Booster	31
4.1.2	Gestione dei Job e Ambiente Offline	33
4.2	Stack Software e Framework di Ottimizzazione	33
4.3	Modelli Linguistici e Dataset Sperimentali	34
4.3.1	Giustificazione delle Scelte di Progetto e Architettureali	35
4.4	Pipeline di Implementazione dei Test	36
4.4.1	Fase 1: Post-Training Quantization Offline	37
4.4.2	Fase 2: Quantization-Aware Training con LoRA	38
4.4.3	Benchmark di Latenza, Throughput e Memoria (<code>evaluate.py</code>)	40
4.4.4	Automazione dei Job tramite Script Batch Slurm	40
5	Risultati Sperimentali e Analisi Comparativa	42
5.1	Protocollo di Valutazione e Metriche di Confronto	42
5.1.1	Inquadramento Scientifico e Significato Applicativo delle Metriche . .	43
5.2	Tabelle dei Risultati Sperimentali su Supercomputer Leonardo	44

5.3	Confronto con i Risultati di Riferimento in Letteratura	46
5.3.1	Aspettative Teoriche dalla Letteratura GPTQ e AWQ	46
5.3.2	Il Fenomeno degli Outlier e la Soglia dei 6.7 Miliardi di Parametri . .	47
5.3.3	QAT e Fine-Tuning a Basso Rango	47
5.4	Nota metodologica: allineamento del ciclo QAT e artefatto di adattamento al dominio	48
5.4.1	Disambiguazione tramite QAT Allineato su Corpus Out-of-Domain (C4)	50
5.5	Analisi del Footprint di Memoria VRAM e Superamento del Memory Wall .	51
5.6	Analisi del Throughput, della Latenza e dell'Accelerazione con <code>torch.compile</code>	52
5.7	Trade-off tra Accuratezza e Metodologia di Quantizzazione	53
6	Conclusioni e Prospettive Future	55
6.1	Sintesi del Lavoro Svolto	55
6.2	Miglioramenti Prestazionali e Analisi Critica dei Limiti	55
6.2.1	Guadagni Prestazionali e Memory Wall	56
6.2.2	Degrado dell'Accuratezza e Limiti Algoritmici	56
6.2.3	Il Collo di Bottiglia Fisico ed Energetico dell'Hardware Convenzionale	57
6.3	La Necessità dell'Interdisciplinarietà per il Superamento dei Limiti	58
6.4	Sostenibilità Ecologica ed Efficienza Energetica (Green AI)	59
6.5	Rilevanza Scientifica, Sociale e Prospettiva Filosofica	59
6.5.1	Democratizzazione dell'Intelligenza Artificiale ed Equità Sociale . . .	59
6.5.2	Dalla Forza Bruta all'Efficienza Algoritmica: Una Prospettiva Episte- mologica	60
6.6	Linee Guida Operative per l'Applicazione della Quantizzazione	60
6.7	Prospettive Future di Ricerca	61
	Bibliografia	63
	Ringraziamenti	68

Elenco delle Figure

2.1	Architettura del modello Transformer introdotta nel lavoro fondazionale “ <i>Attention is All You Need</i> ” [1].	13
3.1	Confronto schematico tra le granularità di quantizzazione applicate a una matrice di pesi. Nella modalità <i>Per-Tensor</i> un unico fattore di scala copre l’intera matrice; nella modalità <i>Per-Channel</i> ogni riga (canale di output) possiede una propria scala; nella modalità <i>Per-Group</i> (usata in questo lavoro con $G = 128$) ogni riga è ulteriormente suddivisa in blocchi contigui, ciascuno con scala indipendente, isolando gli outlier locali a fronte di un modesto overhead di metadati.	22
3.2	Effetto delle strategie di calibrazione sulla scelta dell’intervallo di clipping $[\beta, \alpha]$. La calibrazione <i>Min-Max</i> (linee tratteggiate rosse) include l’intera distribuzione, inclusi i rari outlier: ciò dilata il fattore di scala S (Eq. 3.2) e riduce la risoluzione effettiva riservata alla massa principale dei valori. La calibrazione basata su <i>MSE Minimization</i> (linee continue verdi) tronca deliberatamente gli outlier, restringendo il range e migliorando la risoluzione di quantizzazione per la maggioranza dei pesi, a fronte di un errore di clipping controllato sui pochi valori esclusi.	24
4.1	Vista del supercomputer Leonardo presso il data center CINECA di Bologna.	32
5.1	Throughput medio di generazione (token/s) per ciascuna configurazione. . .	45
5.2	Perplexity misurata sullo split di test di WikiText-2.	45

Elenco delle Tabelle

5.1	<i>Confronto delle prestazioni di inferenza per il modello Meta-Llama-3.1-8B su GPU NVIDIA A100 (64GB) su supercomputer Leonardo.</i>	44
5.2	<i>Confronto delle prestazioni di inferenza per il modello Google Gemma-2-9B su GPU NVIDIA A100 (64GB) su supercomputer Leonardo.</i>	44

Elenco dei Codici

4.1	Procedura di quantizzazione PTQ INT4 con torchao (<code>quantize_ptq.py</code>). . .	37
4.2	Layer di Fake Quantization group-wise coerente con il formato di deployment (<code>train_qat_aligned.py</code>).	39
4.3	Intestazione Slurm comune ai job eseguiti su Leonardo Booster.	41

Capitolo 1

Introduzione e Fondamenti di Machine Learning

1.1 Motivazione e Contesto della Ricerca

Negli ultimi anni, l'Intelligenza Artificiale ha registrato una rapida evoluzione, trainata dalla diffusione dei Modelli Linguistici di Grandi Dimensioni (*Large Language Models*, LLM). Le architetture basate su Transformer mostrano spiccate capacità nella comprensione semantica del testo, nella sintesi di concetti complessi, nel ragionamento logico-matematico, nella generazione di codice e nella traduzione multilingue.

Tuttavia, la capacità espressiva di questi modelli dipende fortemente dalla loro scala dimensionale. Le architetture di frontiera contano regolarmente decine o centinaia di miliardi di parametri, ponendo rilevanti vincoli tecnici ed economici:

1. **Requisiti di Memoria e il “Memory Wall”:** La memorizzazione dei pesi in mezza precisione (FP16 o BF16) richiede 2 Byte per parametro. Un modello da 8 miliardi di parametri necessita di circa 16 GB di memoria video (VRAM) statica, mentre uno da 70 miliardi richiede oltre 140 GB, superando la capacità di una singola GPU di classe datacenter e imponendo l'uso del calcolo distribuito. Inoltre, nella generazione autoregressiva token per token, il continuo trasferimento dei pesi dalla memoria HBM alla SRAM su chip satura la larghezza di banda, rendendo l'inferenza un processo

intrinsecamente limitato dalla memoria (*memory-bound*).

2. **Costi Computazionali ed Energetici:** L'inferenza continua di LLM su larga scala assorbe un'elevata quantità di energia nei data center, determinando alti costi operativi e un impatto ambientale tutt'altro che trascurabile (*Green AI*).
3. **Deployment su Sistemi Embedded ed Edge:** L'integrazione di LLM su dispositivi con risorse limitate (robotica, sistemi IoT, dispositivi mobili) è preclusa senza adeguate strategie di compressione.

In questo contesto, la **Quantizzazione** si impone come la tecnica fondamentale per ridurre l'ingombro in memoria, accelerare l'elaborazione e consentire l'uso efficiente dei modelli di frontiera.

1.2 Obiettivi dell'Elaborato e Contributo Sperimentale

Il presente lavoro di tesi analizza in modo sistematico le moderne tecniche di quantizzazione per modelli linguistici generativi, combinando un'inquadratura teorico-formale a una parte di test e validazione sperimentale.

Gli obiettivi specifici dell'elaborato sono:

- **Formalizzazione Matematica della Quantizzazione:** Analizzare gli schemi di quantizzazione uniforme affine (asimmetrica) e simmetrica, le strategie di calibrazione statistica (Min-Max, MSE, divergenza KL) e i diversi livelli di granularità (per-tensor, per-channel e block-wise a 128 elementi).
- **Studio Comparativo tra PTQ e QAT:** Esaminare i fondamenti della *Post-Training Quantization* (PTQ) focalizzandosi su algoritmi avanzati basati sull'inversa dell'Hessiana quali GPTQ, SmoothQuant e AWQ e del *Quantization-Aware Training* (QAT), approfondendo l'uso dello *Straight-Through Estimator* (STE) e la simulazione di *fake quantization* combinata con la *Low-Rank Adaptation* (LoRA).

- **Indagine sulle Frontiere a 1.58-Bit:** Approfondire architetture emergenti come BitNet b1.58, caratterizzate da pesi ternari $\{-1, 0, 1\}$ che convertono le moltiplicazioni matriciali in sole operazioni di somma e sottrazione.
- **Validazione Sperimentale su Supercomputer Leonardo:** Progettare ed eseguire una pipeline completa di quantizzazione e benchmark su carichi di lavoro reali. La sperimentazione è stata condotta sui nodi della partizione Leonardo Booster del CINECA (equipaggiati con GPU NVIDIA Custom A100 SXM4 da 64GB) resi disponibili dal relatore Prof. Lorenzo Baraldi, impiegando la libreria PyTorch `torchao` e la compilazione JIT tramite `torch.compile`.

1.3 Struttura dell'Elaborato

L'elaborato si articola in sei capitoli:

- Il **Capitolo 1** descrive le motivazioni della ricerca e richiama i fondamenti teorici dell'apprendimento automatico, delle reti neurali profonde, degli acceleratori hardware (GPU e Tensor Cores) e del modello prestazionale Roofline.
- Il **Capitolo 2** illustra l'evoluzione dei modelli di linguaggio, l'architettura Transformer, i meccanismi di attenzione (MHA e GQA), la dinamica di inferenza (prefill e decode) e la gestione della VRAM e della KV Cache.
- Il **Capitolo 3** presenta la formalizzazione matematica della quantizzazione, descrivendo gli algoritmi PTQ (Min-Max, SmoothQuant, GPTQ, AWQ, QuIP), le metodologie QAT (STE, Fake Quantization, LoRA-QAT) e i modelli ternari BitNet b1.58.
- Il **Capitolo 4** illustra l'ambiente di calcolo Leonardo di CINECA, lo stack software (`torchao` e `torch.compile`), i modelli analizzati (*Meta-Llama-3.1-8B* e *Google Gemma-2-9B*), i dataset e la pipeline implementata.
- Il **Capitolo 5** descrive il framework di valutazione e riporta le analisi comparative su latenza (TTFT), throughput (token/s), occupazione di memoria VRAM, fattore di compressione e perplexity.

- Il **Capitolo 6** riassume i risultati ottenuti, discute i compromessi tra efficienza e accuratezza, analizza l'impatto sulla sostenibilità ecologica (Green AI) e traccia le prospettive future.

1.4 Il Paradigma dell'Apprendimento Automatico

L'Apprendimento Automatico (*Machine Learning*, ML) è un pilastro dell'Informatica e dell'Intelligenza Artificiale. A differenza del paradigma di programmazione tradizionale, in cui le regole vengono codificate esplicitamente per trasformare dati in output, nel Machine Learning un sistema apprende relazioni funzionali direttamente dai dati, ottimizzando le proprie prestazioni rispetto a un obiettivo specifico.

Formualmente, dato uno spazio delle istanze $\mathcal{X} \subseteq \mathbb{R}^d$ e uno spazio dei target $\mathcal{Y}$, l'obiettivo dell'apprendimento consiste nell'individuare una funzione $f : \mathcal{X} \rightarrow \mathcal{Y}$, appartenente a una famiglia di ipotesi parametrizzate $\mathcal{H} = \{f(\cdot; \theta) \mid \theta \in \Theta\}$, che minimizzi una funzione di perdita $\mathcal{L}(f(x; \theta), y)$ rispetto alla distribuzione congiunta $P(\mathcal{X}, \mathcal{Y})$. Essendo la vera distribuzione P ignota, si adotta il principio di *Empirical Risk Minimization* (ERM):

$$\theta^* = \arg \min_{\theta \in \Theta} \mathcal{R}_{emp}(\theta) = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(x_i; \theta), y_i) + \lambda \Omega(\theta) \quad (1.1)$$

dove $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ è il dataset di addestramento, $\Omega(\theta)$ è il termine di regolarizzazione e $\lambda \geq 0$ controlla il peso della regolarizzazione per prevenire l'*overfitting*.

I principali paradigmi dell'apprendimento automatico si suddividono in:

- **Apprendimento Supervisionato (*Supervised Learning*)**: il modello apprende da coppie (x_i, y_i) di input ed etichette desiderate per risolvere compiti di classificazione o regressione.
- **Apprendimento Non Supervisionato e Auto-Supervisionato (*Unsupervised / Self-Supervised Learning*)**: il modello analizza dati privi di etichette esplicite per estrarre rappresentazioni dense o distribuite. L'apprendimento auto-supervisionato in particolare costituisce la base dei moderni LLM.

- **Apprendimento per Rinforzo (*Reinforcement Learning*)**: un agente apprende interagendo con un ambiente stocastico mediante azioni, massimizzando il valore atteso di una ricompensa cumulativa scalare.

1.5 Dalle Reti Neurali al Deep Learning

L'oggetto su cui operano concretamente le tecniche di quantizzazione presentate nel Capitolo 3 sono le matrici di pesi che compongono una rete neurale: prima di affrontare la quantizzazione è quindi utile richiamare brevemente da dove tali matrici abbiano origine e come vengano addestrate.

Il *Deep Learning* (DL) estende il modello delle reti neurali artificiali (*Artificial Neural Networks*, ANN) componendo più strati di trasformazioni non lineari. Una rete *feed-forward* a L strati trasforma un vettore in ingresso $x \in \mathbb{R}^{d_0}$ tramite una sequenza di trasformazioni affini e funzioni di attivazione non lineari $\sigma(\cdot)$:

$$h^{(l)} = \sigma \left(W^{(l)} h^{(l-1)} + b^{(l)} \right), \quad l = 1, \dots, L \quad (1.2)$$

dove $h^{(0)} = x$, $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ rappresenta la matrice dei pesi dello strato l , e $b^{(l)} \in \mathbb{R}^{d_l}$ è il vettore di bias. L'impiego di funzioni di attivazione non lineari (come ReLU, GELU, Swish) garantisce la capacità della rete di approssimare funzioni complesse.

L'ottimizzazione dei parametri $\Theta = \{W^{(l)}, b^{(l)}\}_{l=1}^L$ avviene mediante l'algoritmo di *Backpropagation*, che applica la regola della catena per calcolare i gradienti della funzione di perdita:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (h^{(l-1)})^T, \quad \text{con } \delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot \sigma' \left(z^{(l)} \right) \quad (1.3)$$

dove $z^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}$ e $\odot$ indica il prodotto di Hadamard (element-wise). I pesi vengono aggiornati tramite ottimizzatori adattivi come AdamW:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t - \eta \lambda \theta_t \quad (1.4)$$

dove $\hat{m}_t$ e $\hat{v}_t$ sono le stime dei momenti di primo e secondo ordine dei gradienti, η è il *learning rate* e λ è il coefficiente di *weight decay*.

Sono proprio le matrici $W^{(l)}$ appena introdotte nei moderni LLM organizzate in centinaia di strati Transformer (Capitolo 2) e capaci di contenere da alcuni miliardi a centinaia di miliardi di parametri l'oggetto che le tecniche di quantizzazione del Capitolo 3 comprimono a precisione ridotta. Il meccanismo di backpropagation appena descritto tornerà inoltre direttamente in gioco nella Sezione 3.4, dove si mostrerà come l'operatore di arrotondamento della quantizzazione interrompa il flusso dei gradienti qui presentato, e perché sia necessario un accorgimento dedicato (lo *Straight-Through Estimator*) per continuare ad addestrare un modello quando i suoi pesi sono vincolati a una griglia di valori discreti.

1.6 Acceleratori Hardware e Architetture di Calcolo

Lo sviluppo del deep learning è stato sostenuto dall'evoluzione degli acceleratori hardware paralleli, in particolare le *Graphics Processing Units* (GPU) e le *Tensor Processing Units* (TPU).

Mentre le CPU tradizionali sono ottimizzate per l'elaborazione sequenziale ad alta velocità tramite logiche di controllo complesse e memorie cache estese, le GPU adottano un'architettura *many-core* progettata per il calcolo parallelo ad alto throughput (paradigma SIMT, *Single Instruction, Multiple Threads*).

1.6.1 Gerarchia di Memoria e Tensor Cores

Un'architettura moderna per il calcolo scientifico (es. NVIDIA Ampere o Hopper) è suddivisa in decine o centinaia di *Streaming Multiprocessors* (SM). Ogni SM comprende:

1. **ALU Standard (FP32/FP64/INT32):** per operazioni aritmetiche convenzionali scalari e vettoriali.
2. **Tensor Cores:** unità hardware dedicate all'esecuzione in un singolo ciclo di clock di operazioni di moltiplicazione e accumulazione matriciale (*Fused Multiply-Add*, FMA):

$$D = A \times B + C \tag{1.5}$$

dove A e B sono sottomatrici a bassa precisione (es. FP16, BF16, INT8, INT4) e C, D sono accumulatori ad alta precisione (FP32 o INT32).

3. Gerarchia di Memoria Distribuita:

- *Registri locali e Shared Memory / L1 Cache*: memorie SRAM integrate con latenze inferiori al nanosecondo e ampiezze di banda elevate.
- *L2 Cache*: memoria cache condivisa su chip di decine di megabyte.
- *High Bandwidth Memory (HBM/HBM2e/HBM3)*: memoria principale off-chip della GPU (es. 64GB o 80GB con ampiezza di banda tra 1.5 e 3.3 TB/s).

1.7 Valutazione delle Prestazioni: Il Modello Roofline e i Costi Computazionali

Per analizzare le prestazioni dell'inferenza, un modello analitico di riferimento è il **Roofline Model** [2]. Questo modello mette in relazione il picco di calcolo π (FLOP/s) e la larghezza di banda della memoria β (Byte/s) con l'*Intensità Operativa* I , definita come il numero di operazioni in virgola mobile eseguite per byte trasferito:

$$I = \frac{\text{Operazioni Aritmetiche [FLOPs]}}{\text{Dati Trasferiti [Bytes]}} \quad \left[\frac{\text{FLOP}}{\text{Byte}} \right] \quad (1.6)$$

Il throughput massimo teorico P è vincolato da:

$$P = \min(\pi, \beta \times I) \quad (1.7)$$

Tale relazione individua due regimi di funzionamento:

1. **Regime Memory-Bound** ($I < \frac{\pi}{\beta}$): il throughput è limitato dalla velocità di trasferimento dei dati dalla memoria HBM alla SRAM su chip. In questa condizione, ridurre la precisione dei parametri (es. da 16 a 4 bit) riduce l'occupazione di banda e incrementa la velocità di generazione a parità di operazioni aritmetiche.
2. **Regime Compute-Bound** ($I \geq \frac{\pi}{\beta}$): il sistema viene limitato dalla capacità di calcolo aritmetico degli SM e dei Tensor Cores.

Le metriche principali adottate nella sperimentazione di questa tesi sono:

- **Time-To-First-Token (TTFT)**: latenza di elaborazione del prompt di ingresso e generazione del primo token (fase di *prefill*).
- **Throughput di Generazione (Token/s)**: frequenza di generazione dei token durante la fase autoregressiva (fase di *decode*).
- **Peak Memory Footprint (VRAM GiB)**: picco massimo di memoria allocata sulla GPU.
- **Fattore di Compressione**: rapporto tra la dimensione del modello in precisione di riferimento e quella del modello quantizzato.
- **Perplexity**: misura della capacità del modello di predire il testo, calcolata sullo split di test di WikiText-2.

Capitolo 2

I Modelli Linguistici di Grandi Dimensioni (LLM)

2.1 Cos'è un Modello Linguistico: Definizione, Classificazione e Relazione con la Quantizzazione

Prima di ripercorrerne l'evoluzione storica, è opportuno chiarire cosa si intenda, in generale, per **Modello Linguistico** (*Language Model*, LM). Un modello linguistico è, nella sua accezione più ampia, un modello statistico che assegna una probabilità a una sequenza di token $x_1, \dots, x_T$ appresi da un vocabolario finito $\mathcal{V}$. Sfruttando la regola della catena della probabilità, tale probabilità congiunta viene fattorizzata in un prodotto di probabilità condizionate, ciascuna dipendente solo dai token che la precedono:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}) \quad (2.1)$$

Questa formulazione non serve soltanto a “valutare” quanto una frase sia plausibile (ad esempio per stimare la *perplexity*, la metrica usata estensivamente nel Capitolo 5 per quantificare il degrado introdotto dalla quantizzazione), ma consente anche di *generare* nuovo testo, campionando iterativamente $x_t \sim P(\cdot \mid x_{<t})$: è esattamente questo processo iterativo, chiamato *decode autoregressivo*, a essere analizzato nella Sezione 2.4 e a costituire il collo di bottiglia prestazionale che la quantizzazione mira a mitigare.

La letteratura sul trattamento automatico del linguaggio individua tipicamente quattro grandi categorie di modelli linguistici, corrispondenti anche a quattro tappe storiche successive:

1. **Modelli Linguistici Statistici (*Statistical Language Models*, SLM)**, tra cui i modelli a N -grammi: stimano la probabilità di un token w_t condizionata ai precedenti $N - 1$ token sulla base delle frequenze di co-occorrenza osservate nel corpus:

$$P(w_t \mid w_{t-N+1}, \dots, w_{t-1}) \approx \frac{\text{Count}(w_{t-N+1}, \dots, w_t)}{\text{Count}(w_{t-N+1}, \dots, w_{t-1})} \quad (2.2)$$

Questi modelli soffrivano di dispersione statistica (*data sparsity*) e dell'incapacità di modellare dipendenze a lungo raggio o relazioni semantiche.

2. **Modelli Linguistici Neurali (*Neural Language Models*, NLM)**, tra cui le reti ricorrenti (RNN / LSTM / GRU): hanno introdotto i vettori densi nello spazio continuo (*word embeddings*) e gli stati nascosti ricorsivi $h_t = \sigma(Wx_t + Uh_{t-1} + b)$ per elaborare sequenze di lunghezza variabile. La natura sequenziale delle ricorrenze impediva tuttavia la parallelizzazione dell'addestramento su GPU e soffriva del problema del gradiente evanescente (*vanishing gradient*) su contesti estesi.

3. **Modelli Linguistici Pre-addestrati (*Pre-trained Language Models*, PLM)**: con l'introduzione dell'architettura Transformer [1] (Sezione 2.2) si è affermato un nuovo paradigma in due fasi, il *pre-training* seguito dal *fine-tuning*: un modello viene prima addestrato su grandi corpora testuali non etichettati per apprendere rappresentazioni linguistiche generali, e successivamente specializzato con un addestramento supervisionato di dimensioni contenute su un compito specifico. Modelli come BERT [3] (encoder-only) e la prima generazione di GPT hanno dimostrato l'efficacia di questo approccio di *transfer learning* applicato al linguaggio.

4. **Grandi Modelli Linguistici (*Large Language Models*, LLM)**: portando il paradigma dei PLM a una scala di miliardi o centinaia di miliardi di parametri, addestrati su corpora dell'ordine dei trilioni di token, emergono capacità qualitativamente nuove e non esplicitamente insegnate, dette *emergent abilities* (*few-shot prompting*, *in-context*

learning, ragionamento a catena), che permettono al modello di affrontare compiti nuovi senza un fine-tuning dedicato, semplicemente tramite istruzioni ed esempi forniti nel prompt. Modelli come GPT-3 [4], LLaMA [5, 6] e Gemma [7], oggetto di questa tesi, appartengono a questa quarta categoria.

È proprio in corrispondenza di questa quarta categoria che nasce, in modo naturale, la necessità della quantizzazione discussa nel Capitolo 3. Le capacità emergenti dei LLM sono infatti indissolubilmente legate alla loro scala dimensionale (Sezione 1.1 del Capitolo 1): è proprio questa scala, e non un difetto architetturale, a generare il problema del *Memory Wall* e i costi computazionali ed energetici illustrati nell'introduzione. Per i modelli statistici, neurali e pre-addestrati di generazione precedente, tipicamente di dimensioni contenute (milioni anziché miliardi di parametri), la quantizzazione a bassa precisione non costituisce un'esigenza pratica stringente; per gli LLM, invece, è proprio la scala a rendere la compressione numerica una condizione quasi necessaria per un deployment sostenibile. A ciò si aggiunge una seconda ragione, di natura più scientifica che ingegneristica: i LLM risultano fortemente sovra-parametrizzati rispetto alla quantità di informazione realmente necessaria a svolgere un singolo compito, un fenomeno di ridondanza rappresentazionale che, come mostreranno le formalizzazioni del Capitolo 3 e i risultati sperimentali del Capitolo 5, consente di ridurre la precisione numerica dei pesi fino a 4 bit (e in alcuni casi fino a valori ternari, Sezione 3.5) preservando gran parte della capacità linguistica del modello.

2.2 L'Architettura Transformer e i Meccanismi di Attenzione

Questa sezione descrive nel dettaglio l'architettura interna dei LLM introdotti come quarta categoria nella Sezione 2.1, poiché è proprio all'interno di questi blocchi che si trovano le matrici di pesi (W_Q , W_K , W_V , W_O e le matrici della rete Feed-Forward) su cui viene applicata la quantizzazione discussa nel Capitolo 3: comprendere quali proiezioni lineari compongono un blocco Transformer, e quanto ciascuna pesi in termini di parametri, è un prerequisito per

interpretare correttamente sia la scelta dei moduli LoRA nel QAT (Sezione 3.4), sia i risultati di compressione della memoria riportati nel Capitolo 5.

Il blocco fondamentale del Transformer è il modulo di **Scaled Dot-Product Attention**. Data una sequenza di input rappresentata dalla matrice $X \in \mathbb{R}^{T \times d_{model}}$, vengono definite tre proiezioni lineari: *Query* (Q), *Key* (K) e *Value* (V):

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (2.3)$$

con matrici di peso $W_Q, W_K \in \mathbb{R}^{d_{model} \times d_k}$ e $W_V \in \mathbb{R}^{d_{model} \times d_v}$. L'attenzione è definita come:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V \quad (2.4)$$

dove $\sqrt{d_k}$ è il fattore di riscalamento introdotto per evitare che i prodotti scalari raggiungano magnitudini elevate spingendo il gradiente della Softmax in regioni di saturazione. La matrice $M \in \mathbb{R}^{T \times T}$ rappresenta la maschera causale (*causal mask*): nei modelli autoregressivi, $M_{i,j} = -\infty$ per $j > i$ e 0 altrimenti, garantendo che ciascun token acceda unicamente ai token precedenti.

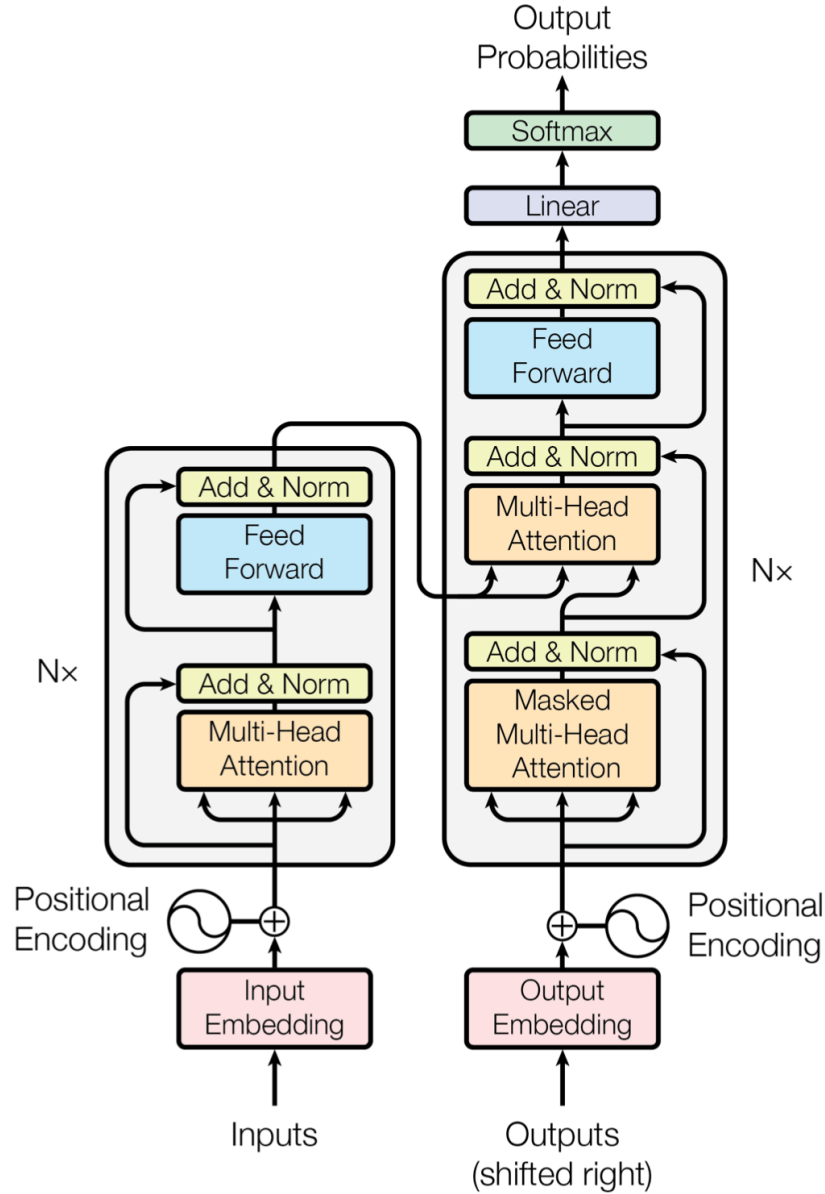


Figura 2.1: Architettura del modello Transformer introdotta nel lavoro fondazionale “*Attention is All You Need*” [1].

La Figura 2.1 riporta l’architettura originale, composta da uno stack Encoder (a sinistra) e uno stack Decoder (a destra) collegati da un meccanismo di *cross-attention*. È importante precisare che Meta-Llama-3.1-8B e Google Gemma-2-9B, i due modelli su cui verte la sperimentazione di questa tesi (Capitolo 4), sono architetture **Decoder-Only** (Sezione 2.3): utilizzano quindi solo il ramo destro dello schema, senza encoder né cross-attention, e con

la sola *Masked Self-Attention* già formalizzata in Eq. 2.4 tramite la maschera causale M . La figura viene comunque mostrata nella sua forma completa e originale perché resta il riferimento storico da cui derivano tutte le equazioni di questa sezione (proiezioni Q, K, V , attenzione multi-testa, blocco Feed-Forward), ed è utile per collocare correttamente l'architettura Decoder-Only, oggetto di questa tesi, all'interno del quadro più ampio della famiglia Transformer.

Nel meccanismo di **Multi-Head Attention** (MHA), le proiezioni vengono replicate su h teste di attenzione indipendenti per catturare informazioni da diversi sottospazi di rappresentazione:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \quad (2.5)$$

$$\text{con } \text{head}_i = \text{Attention}(QW_{Q,i}, KW_{K,i}, VW_{V,i}) \quad (2.6)$$

Nelle architetture moderne come LLaMA-3 [6] e Gemma-2 [7], l'attenzione Multi-Head è affiancata dalla **Grouped-Query Attention** (GQA), in cui più teste Query condividono una singola coppia di teste Key-Value, riducendo l'occupazione di memoria della *KV Cache* durante l'inferenza. Questo accorgimento architetturale agisce sullo stesso problema del *Memory Wall* discusso nel Capitolo 1, ma a un livello complementare rispetto alla quantizzazione: mentre la GQA riduce staticamente la dimensione della cache Key-Value, la quantizzazione (Capitolo 3) riduce la precisione dei pesi statici del modello; l'estensione della quantizzazione alla KV Cache stessa, non affrontata nella parte sperimentale di questa tesi, è indicata come prospettiva futura nella Sezione 6.7.

2.2.1 Feed-Forward Networks e Varianti Moderne

Ciascun blocco Transformer integra uno strato *Feed-Forward Network* (FFN) operante in modo indipendente su ciascuna posizione. Nei modelli moderni, la combinazione lineare classica è spesso sostituita dalla variante *SwiGLU* (*Swish Gated Linear Unit*):

$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{Swish}(xW_{\text{gate}}) \odot (xW_{\text{up}})) W_{\text{down}} \quad (2.7)$$

Questo modulo racchiude circa i due terzi dei parametri totali di un LLM standard, rappresentando il bersaglio principale per le tecniche di quantizzazione.

2.3 Tassonomia dei Modelli: Dai PLM agli LLM Generativi

Oltre alla classificazione storica in quattro categorie appena presentata (Sezione 2.1), i modelli basati su Transformer possono essere classificati anche secondo un asse complementare, di natura puramente architetturale, che distingue tre famiglie in base a come il meccanismo di attenzione viene impiegato:

1. **Encoder-Only** (es. **BERT** [3], **RoBERTa**): utilizzano attenzione bidirezionale e sono pre-addestrati tramite *Masked Language Modeling* (MLM). Trovano impiego in compiti di classificazione ed estrazione di informazioni.
2. **Encoder-Decoder** (es. **T5**, **BART**): combinano un encoder bidirezionale con un decoder causale, per task sequenza-a-sequenza come traduzione e sintesi.
3. **Decoder-Only / Causal LLM** (es. **GPT-3** [4], **LLaMA** [5, 6], **Gemma** [7]): adottano un'architettura autoregressiva addestrata per massimizzare la verosimiglianza del token successivo (*Next-Token Prediction*):

$$\mathcal{L}_{autoreg}(\theta) = - \sum_{t=1}^T \log P(x_t \mid x_{<t}; \theta) \quad (2.8)$$

Con l'aumento della scala dei parametri (*Scaling Laws*), i Causal LLM hanno mostrato capacità emergenti come *few-shot prompting*, *in-context learning* e generazione di codice. Meta-Llama-3.1-8B e Google Gemma-2-9B, i due modelli quantizzati nella parte sperimentale di questa tesi (Capitolo 4), appartengono entrambi a questa famiglia Decoder-Only.

2.4 La Dinamica di Inferenza: Prefill, Decode e la KV Cache

L'inferenza di un LLM si articola in due fasi computazionali distinte:

1. Fase di Prefill (Prompt Processing):

- L'intero prompt di input (lunghezza T_{in}) viene elaborato in parallelo.
- Le moltiplicazioni matriciali sono eseguite tramite operazioni Dense GEMM (*General Matrix Multiplication*).
- L'intensità operativa I è alta; questa fase risulta prevalentemente **Compute-Bound**.
- Al termine della fase, i valori K e V del contesto vengono salvati nella *KV Cache*.

2. Fase di Decode (Token-by-Token Generation):

- I token vengono generati in modo sequenziale, un token per passo temporale t .
- Ad ogni passo, la generazione del singolo token richiede il caricamento di tutti i pesi del modello.
- Le moltiplicazioni matriciali si riducono a operazioni GEMV (*General Matrix-Vector Multiplication*) con batch size pari a 1.
- L'intensità operativa scende a $I \ll 1$ FLOP/Byte; questa fase è strettamente **Memory-Bound**.
- La velocità di generazione è limitata dalla larghezza di banda della memoria della GPU.

2.5 I Limiti Computazionali e il Problema della Memoria

Le dimensioni dei moderni LLM impongono requisiti di memoria elevati per il deployment. L'occupazione di memoria VRAM richiesta per allocare un modello con P parametri in precisione a 16 bit (FP16 o BF16, 2 Byte per parametro) è pari a:

$$\text{Memoria Pesi (FP16)} = P \times 2 \text{ Bytes} \quad (2.9)$$

A titolo esemplificativo:

- Un modello da 8B parametri (es. Meta-Llama-3.1-8B) richiede circa 16 GB per i soli pesi statici.
- Un modello da 70B parametri necessita di oltre 140 GB, superando la capacità di una singola GPU datacenter (es. NVIDIA A100 80GB) e richiedendo strategie di parallelismo distribuito (*Tensor Parallelism*).
- Architetture da 175B parametri richiedono fino a 350 GB di VRAM.

A questo contributo si aggiunge la memoria della **KV Cache**, che scala linearmente con la lunghezza della sequenza S , il numero di layer L , il numero di teste KV h_{kv} , la dimensione della testa d_k e la dimensione del batch B :

$$\text{Memoria KV Cache} = 2 \times 2 \times B \times L \times h_{kv} \times d_k \times S \text{ Bytes} \quad (2.10)$$

Per contesti estesi (32k o 128k token), la sola KV Cache richiede diverse decine di gigabyte.

In questo scenario, la **Quantizzazione** rappresenta la soluzione fondamentale per ridurre l'impronta di memoria, superare la limitazione del Memory Wall e consentire l'esecuzione efficiente dei modelli linguistici.

Capitolo 3

Teoria e Tecniche di Quantizzazione

3.1 Rappresentazione Numerica nei Sistemi di Calcolo

Nei sistemi di calcolo per il deep learning, la scelta del formato di rappresentazione numerica non è un dettaglio implementativo marginale: determina direttamente quanto un modello sia grande, quanto velocemente possa rispondere e su quale hardware possa essere effettivamente eseguito. Precisione aritmetica, consumo di memoria, larghezza di banda e velocità di elaborazione sono infatti tra loro in stretto compromesso [8–10], e ogni scelta di formato numerico rappresenta un punto diverso lungo questo compromesso.

Per lungo tempo l’addestramento e l’inferenza delle reti neurali sono stati condotti quasi esclusivamente in virgola mobile a singola precisione a 32 bit, lo standard IEEE 754 noto come **FP32** (1 bit di segno s , 8 bit di esponente e , 23 bit di mantissa m): un formato scelto storicamente per l’ampio margine di sicurezza numerica che offre, a costo però di 4 byte per ogni singolo parametro. Quando i modelli hanno iniziato a crescere fino a miliardi di parametri, questo costo è diventato rapidamente insostenibile, ed è per questo che l’industria si è spostata verso formati a 16 bit, dimezzando la memoria necessaria senza dover ripensare l’intera aritmetica di calcolo. Non tutti i formati a 16 bit sono però equivalenti tra loro. Il **FP16 (Half Precision)** dedica 5 bit all’esponente e 10 alla mantissa: offre una buona precisione locale, ma il suo intervallo dinamico ridotto (all’incirca tra 10^{-5} e 6.5×10^4) può facilmente generare fenomeni di *overflow* o *underflow* durante l’addestramento, quando i gradienti assumono valori molto piccoli o molto grandi. Il **BF16 (Brain Floating Point)**

risolve questo problema in modo elegante: sacrifica precisione nella mantissa (appena 7 bit) per mantenere lo stesso numero di bit di esponente di FP32 (8 bit, e dunque lo stesso intervallo dinamico, fino a circa 10^{38}). È proprio questa scelta a renderlo lo standard *de facto* per l'addestramento e il deployment dei moderni LLM, incluse le due architetture analizzate in questa tesi, LLaMA-3 e Gemma-2.

La quantizzazione spinge lo stesso principio ben oltre i 16 bit, rappresentando i valori come numeri interi anziché in virgola mobile. Il formato **INT8** colloca ogni valore in un intervallo discreto di soli 256 livelli ($[-128, 127]$, o simmetricamente $[-127, 127]$): è già sufficiente per molti compiti di visione artificiale, ma nei grandi modelli linguistici, caratterizzati da miliardi di parametri fortemente ridondanti, si può spingere la compressione ancora oltre. Il formato **INT4**, impiegato nella parte sperimentale di questo lavoro, mappa ogni peso su appena 16 livelli discreti (intervallo $[-8, 7]$ oppure $[0, 15]$): rappresenta, come emergerà anche dai risultati sperimentali del Capitolo 5, il punto di equilibrio più diffuso in letteratura tra riduzione di memoria (fino a $4\times$ rispetto a BF16) e qualità del modello preservata. Oltre questo limite si collocano infine i formati sub-4-bit e ternari, che riducono ciascun peso a soli 2 bit (4 livelli) o a un insieme di appena tre valori $\{-1, 0, 1\}$ (rappresentabile in teoria con $\log_2(3) \approx 1.58$ bit per parametro): si tratta di regimi ancora sperimentali, discussi nella Sezione 3.5, che richiedono tecniche di addestramento dedicate per non compromettere irrimediabilmente la qualità del modello.

3.2 Formalizzazione Matematica della Quantizzazione

Spogliata delle formule, la quantizzazione è in fondo un'operazione di arrotondamento intelligente: un modo per rappresentare un insieme continuo (o comunque molto fine) di valori reali $X \in \mathbb{R}$ tramite un insieme finito e discreto di valori $X_Q \in \mathcal{Q}$, scegliendo la corrispondenza tra i due insiemi in modo da perdere quanta meno informazione possibile.

3.2.1 Quantizzazione Uniforme Affine (Asimmetrica)

L'idea alla base della **Quantizzazione Affine** è semplice e intuitiva: si osserva l'intervallo di valori realmente assunto dal tensore da quantizzare, $[\beta, \alpha] = [\min(X), \max(X)]$, e lo si fa

corrispondere in modo lineare all'intervallo di interi rappresentabili con B bit, ad esempio $[0, 2^B - 1]$ per interi unsigned. In pratica, la retta reale osservata viene “srotolata” su una griglia di interi equispaziati. La funzione di quantizzazione diretta è definita da:

$$X_Q = \text{clip} \left(\left\lfloor \frac{X}{S} \right\rfloor + Z, q_{min}, q_{max} \right) \quad (3.1)$$

dove $\lfloor \cdot \rfloor$ indica l'arrotondamento all'intero più vicino (*round-to-nearest*) e $\text{clip}(v, l, u) = \max(l, \min(v, u))$ evita che eventuali valori fuori range producano interi non rappresentabili. I due parametri che governano la mappatura sono il fattore di scala $S \in \mathbb{R}^+$ (*scale factor*) e il punto zero $Z \in \mathbb{Z}$ (*zero-point*):

$$S = \frac{\alpha - \beta}{q_{max} - q_{min}} = \frac{\alpha - \beta}{2^B - 1} \quad (3.2)$$

$$Z = \text{round} \left(-\frac{\beta}{S} \right) + q_{min} \quad (3.3)$$

Intuitivamente, S rappresenta quanto valore reale è racchiuso in ciascun “gradino” della griglia intera (più S è piccolo, più la griglia è fine e precisa), mentre Z è un semplice offset intero che sposta la griglia in modo che il valore reale 0.0 cada esattamente su un intero rappresentabile, invece che a metà tra due gradini. Per rendere concreto il calcolo, si consideri un tensore di attivazioni il cui intervallo osservato sia $[\beta, \alpha] = [-2.0, 6.0]$, da quantizzare a INT8 unsigned ($q_{min} = 0, q_{max} = 255$): applicando le equazioni precedenti si ottiene $S = (6.0 - (-2.0))/255 \approx 0.0314$ e $Z = \text{round}(2.0/0.0314) = 64$. Lo zero reale verrebbe quindi codificato esattamente dall'intero 64, senza alcun errore di arrotondamento su di esso: un dettaglio tutt'altro che secondario, perché lo zero ricorre molto spesso nelle attivazioni delle reti neurali (padding delle sequenze, mascheramento causale, unità ReLU). Il percorso inverso, la **dequantizzazione**, ricostruisce un valore approssimato $\hat{X} \approx X$ a partire dall'intero memorizzato:

$$\hat{X} = S \cdot (X_Q - Z) \quad (3.4)$$

3.2.2 Quantizzazione di Scala Simmetrica

Quando la distribuzione dei valori da quantizzare è già approssimativamente centrata sullo zero — come accade tipicamente per i pesi di una rete neurale, la cui distribuzione risulta

pressoché simmetrica per effetto dell’inizializzazione e della regolarizzazione durante l’addestramento è possibile semplificare lo schema precedente eliminando del tutto il punto zero. La **Quantizzazione Simmetrica** assume un intervallo reale simmetrico $[-\alpha, \alpha]$, dove $\alpha = \max(|X|)$, e vincola $Z = 0$:

$$S = \frac{\alpha}{2^{B-1} - 1} \quad (3.5)$$

$$X_Q = \text{clip} \left(\left\lfloor \frac{X}{S} \right\rfloor, -2^{B-1} + 1, 2^{B-1} - 1 \right) \quad (3.6)$$

$$\hat{X} = S \cdot X_Q \quad (3.7)$$

Questa semplificazione non è solo estetica: ha un impatto diretto sulle prestazioni hardware. Nelle moltiplicazioni matriciali tra attivazioni quantizzate X_Q e pesi quantizzati W_Q , il punto zero introdurrebbe termini incrociati di correzione ($Z_X W_Q + Z_W X_Q$) da calcolare e sommare per ogni singolo prodotto scalare, un costo aggiuntivo non trascurabile su matrici di miliardi di elementi. Ponendo $Z = 0$, tali termini scompaiono del tutto e la moltiplicazione può essere eseguita direttamente in aritmetica intera, con il riscalamento in virgola mobile applicato una sola volta al risultato finale:

$$\hat{Y} = \hat{X} \hat{W} = (S_X X_Q)(S_W W_Q) = (S_X S_W) \cdot (X_Q W_Q) \quad (3.8)$$

È questo il motivo per cui, nella pratica, la quantizzazione simmetrica è preferita per i pesi (dove il punto zero non offre benefici significativi) mentre la variante asimmetrica resta utile per le attivazioni post-ReLU o post-GeLU, che assumono valori tipicamente non centrati sullo zero.

3.2.3 Granularità della Quantizzazione

Le equazioni precedenti calcolano un’unica coppia (S, Z) per l’intero tensore considerato, ma questa non è l’unica scelta possibile: la **granularità** definisce quante coppie (S, Z) vengono effettivamente calcolate e su quale porzione del tensore ciascuna di esse viene applicata, ed è in pratica la leva più importante per controllare il compromesso tra qualità e overhead di memoria. Nella granularità **Per-Tensor** un singolo fattore di scala copre l’intera matrice dei pesi o delle attivazioni: l’overhead di memoria è minimo (un solo numero da memorizzare per l’intero tensore), ma se anche un solo canale della matrice contiene valori molto più

grandi degli altri, quel canale “detta legge” sulla scala S dell’intero tensore, penalizzando la risoluzione di tutti gli altri. La granularità **Per-Channel** (o *Per-Token* per le attivazioni) risolve in parte questo problema calcolando un fattore di scala indipendente per ciascuna riga della matrice dei pesi, cioè per ciascun canale di output ($S \in \mathbb{R}^{d_{out}}$): è lo standard de facto per la quantizzazione INT8 dei pesi, perché l’overhead resta contenuto (un numero per riga, anziché uno per l’intero tensore) mentre la precisione migliora sensibilmente. Quando però si scende a 4 bit, anche la variabilità *all’interno* di una singola riga può diventare eccessiva, ed è qui che entra in gioco la granularità **Group-wise** (o *Block-wise*): ciascuna riga viene ulteriormente suddivisa in blocchi contigui di dimensione fissa G (tipicamente $G = 128$ o $G = 64$), ciascuno con il proprio fattore di scala S_g . In questo lavoro, in linea con la configurazione `Int4WeightOnlyConfig` di `torchao` impiegata nella parte sperimentale (Capitolo 4), si adotta $G = 128$: per una tipica proiezione lineare 4096×4096 , questo significa calcolare $4096 \times (4096/128) = 131,072$ fattori di scala anziché uno solo, con un overhead di memoria di poche centinaia di kilobyte a fronte di una matrice di pesi che occupa diversi megabyte in INT4: un costo del tutto trascurabile rispetto al beneficio in accuratezza che ne deriva, poiché ogni blocco può adattarsi localmente alla presenza di outlier senza distorcere la quantizzazione dell’intera riga.

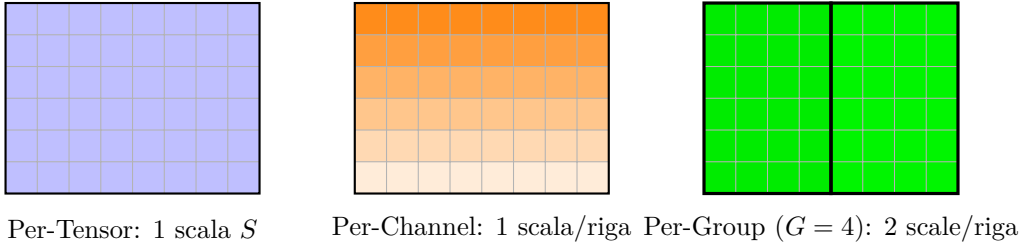


Figura 3.1: Confronto schematico tra le granularità di quantizzazione applicate a una matrice di pesi. Nella modalità *Per-Tensor* un unico fattore di scala copre l’intera matrice; nella modalità *Per-Channel* ogni riga (canale di output) possiede una propria scala; nella modalità *Per-Group* (usata in questo lavoro con $G = 128$) ogni riga è ulteriormente suddivisa in blocchi contigui, ciascuno con scala indipendente, isolando gli outlier locali a fronte di un modesto overhead di metadati.

3.2.4 Metodologie di Calibrazione

Una volta stabilita la granularità, resta da decidere *come* individuare i confini $[\beta, \alpha]$ da usare nelle Eq. 3.2-3.3: questo è il compito della **calibrazione**, condotta osservando le attivazioni o i pesi su un piccolo campione di dati rappresentativo (*Calibration Dataset*), tipicamente poche centinaia di esempi. L’approccio più immediato, il **Min-Max Search**, si limita a registrare il valore minimo e massimo assoluto osservati nel campione: è il metodo adottato ad esempio dalla configurazione **torchao** usata per la PTQ Min-Max in questa tesi (Capitolo 4), perché immediato da calcolare e privo di iperparametri, ma per costruzione estremamente sensibile agli outlier: un singolo valore anomalo osservato durante la calibrazione può dilatare inutilmente l’intero intervallo $[\beta, \alpha]$, penalizzando la risoluzione di tutti i valori “normali”. Per mitigare questo problema, la calibrazione basata sulla **Minimizzazione dell’Errore Quadratico Medio (MSE)** non usa semplicemente gli estremi osservati, ma cerca la soglia di clipping α che minimizza la distanza tra il tensore originale e la sua versione quantizzata:

$$\alpha^* = \arg \min_{\alpha} \|X - \hat{X}(S(\alpha))\|_2^2 \quad (3.9)$$

In altre parole, l’algoritmo accetta consapevolmente di troncare (*clippare*) i pochi valori più estremi, sacrificando la loro precisione, a patto di guadagnare una risoluzione molto migliore su tutta la massa principale della distribuzione, come illustrato in Figura 3.2. Una terza strategia, storicamente impiegata ad esempio nel motore di inferenza NVIDIA TensorRT per la calibrazione INT8, si basa sulla **Divergenza di Kullback-Leibler (KL)**: anziché minimizzare un errore quadratico, cerca il range che minimizza la divergenza informativa tra la distribuzione continua originale P e quella discretizzata Q :

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)} \quad (3.10)$$

risultando particolarmente efficace quando la distribuzione dei valori è fortemente non gaussiana. Nella pratica sperimentale di questa tesi, come si vedrà nel Capitolo 4, alla calibrazione Min-Max si affianca l’algoritmo GPTQ (Sezione 3.3), che spinge il principio della calibrazione ancora oltre, non limitandosi a scegliere un buon range ma correggendo attivamente i pesi non ancora quantizzati per compensare l’errore già introdotto.

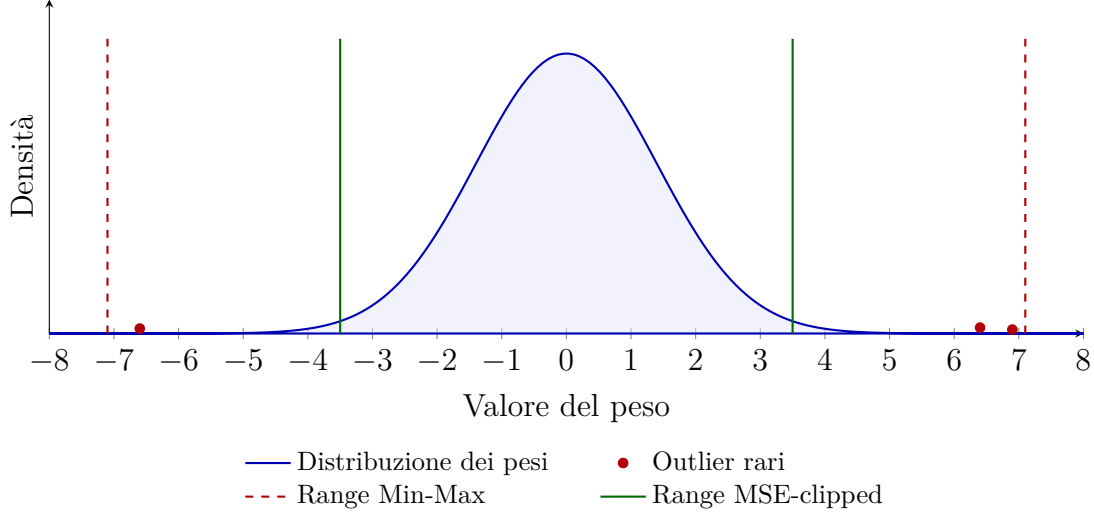


Figura 3.2: Effetto delle strategie di calibrazione sulla scelta dell’intervallo di clipping $[\beta, \alpha]$. La calibrazione *Min-Max* (linee tratteggiate rosse) include l’intera distribuzione, inclusi i rari outlier: ciò dilata il fattore di scala S (Eq. 3.2) e riduce la risoluzione effettiva riservata alla massa principale dei valori. La calibrazione basata su *MSE Minimization* (linee continue verdi) tronca deliberatamente gli outlier, restringendo il range e migliorando la risoluzione di quantizzazione per la maggioranza dei pesi, a fronte di un errore di clipping controllato sui pochi valori esclusi.

3.3 Post-Training Quantization (PTQ)

La **Post-Training Quantization** (PTQ) converte un modello pre-addestrato ad alta precisione in un formato a bassa precisione senza richiedere un ri-addestramento completo dei pesi, risultando efficiente in termini di tempo e risorse computazionali.

3.3.1 Weight-Only vs Weight-Activation Quantization

Non tutto, in un modello linguistico, deve necessariamente essere quantizzato allo stesso modo: una prima distinzione pratica, che ritornerà più volte nel seguito di questa tesi, riguarda *cosa* venga effettivamente compresso a bassa precisione. Nello schema **Weight-Only Quantization** (indicato con la notazione W4A16 o W8A16) si quantizzano soltanto le matrici dei pesi, che sono statiche e note in anticipo, mentre le attivazioni e i calcoli restano in FP16 o

BF16. In fase di inferenza i pesi a 4 bit vengono letti dalla memoria HBM, trasferiti nella SRAM della GPU e dequantizzati “al volo” (*on-the-fly dequantization*) subito prima della moltiplicazione: poiché nella fase di *decode* autoregressiva degli LLM il collo di bottiglia è proprio il trasferimento dei pesi dalla memoria (Sezione 5.5 del Capitolo 5), ridurre la sola dimensione dei pesi è già sufficiente a ottenere gran parte del beneficio, senza dover affrontare i rischi di instabilità numerica che la quantizzazione delle attivazioni comporterebbe: è questo l’approccio adottato in tutta la parte sperimentale di questa tesi (Capitolo 4). Lo schema **Weight-Activation Quantization** (W8A8 o W4A4) spinge la compressione anche sulle attivazioni dinamiche, generate ad ogni passo in funzione dell’input: farlo consente di eseguire l’intera moltiplicazione matrice-matrice in aritmetica intera sui Tensor Cores dedicati, accelerando in particolare la fase di *prefill* (dove il collo di bottiglia è il calcolo, non la memoria), ma richiede di gestire con cura le attivazioni, che variano ad ogni input e sono, come si vedrà a breve, molto più soggette alla presenza di outlier rispetto ai pesi.

3.3.2 Il Problema degli Outlier di Attivazione e SmoothQuant

Un ostacolo pratico e ricorrente nella quantizzazione dei grandi modelli linguistici è la comparsa di valori anomali (*outlier*) nelle attivazioni: Dettmers et al. [14] hanno osservato che, al crescere delle dimensioni del modello oltre i 6.7B parametri, alcuni canali di attivazione assumono sistematicamente valori fino a 100 volte superiori alla media degli altri canali. Non si tratta di un artefatto casuale: questi canali tendono a essere sempre gli stessi attraverso input diversi, e a codificare informazione semantica rilevante che il modello ha imparato a enfatizzare. Il problema, dal punto di vista della quantizzazione, è che un singolo canale anomalo costringe l’intero tensore ad adottare un fattore di scala S molto più grande del necessario, schiacciando la risoluzione riservata a tutti gli altri canali “normali”.

SmoothQuant [15] propone una soluzione elegante a questo problema, sfruttando una semplice osservazione algebrica: se si moltiplica ogni colonna delle attivazioni per un fattore e si divide la corrispondente riga dei pesi per lo stesso fattore, il risultato della moltiplicazione $Y = XW$ non cambia, ma la difficoltà di quantizzazione può essere *trasferita* da un operando

all'altro. Introducendo una matrice diagonale di scala $S = \text{diag}(s)$:

$$Y = (X \cdot \text{diag}(s)^{-1}) \cdot (\text{diag}(s) \cdot W) = \hat{X}\hat{W} \quad (3.11)$$

e scegliendo il fattore di *smoothing* per ciascun canale j in modo da bilanciare la magnitudine osservata su attivazioni e pesi,

$$s_j = \frac{\max(|X_j|)^\alpha}{\max(|W_j|)^{1-\alpha}}, \quad \alpha \in [0, 1] \quad (3.12)$$

SmoothQuant sposta parte della difficoltà dagli outlier di attivazione (difficili da quantizzare) verso i pesi (naturalmente più uniformi e quindi più tolleranti alla compressione), rendendo così percorribile la quantizzazione W8A8 senza una perdita di accuratezza significativa.

3.3.3 Algoritmi PTQ Avanzati di Secondo Ordine: GPTQ e AWQ

Quando si scende ulteriormente a 4 bit, calibrare semplicemente un buon range non basta più: servono algoritmi in grado di *correggere attivamente* l'errore introdotto dalla quantizzazione. **GPTQ** [16] affronta il problema riprendendo l'idea dell'*Optimal Brain Surgeon* (OBS), un metodo storicamente usato per il pruning delle reti neurali: l'obiettivo è minimizzare l'errore quadratico introdotto sull'output dello strato, $\arg \min_{\hat{W}} \|WX - \hat{W}X\|_2^2$, sfruttando l'informazione di secondo ordine contenuta nella matrice Hessiana $H = 2XX^T$, che descrive quanto ciascun peso influenzi l'output rispetto agli altri. L'idea pratica è di procedere colonna per colonna: ogni volta che una colonna di pesi viene quantizzata, l'errore commesso non viene semplicemente ignorato, ma redistribuito sulle colonne non ancora quantizzate, in modo che possano compensarlo:

$$\hat{w}_q = \text{quant}(w_q), \quad \Delta W_{:, > q} = -\frac{w_q - \hat{w}_q}{[H^{-1}]_{qq}} \cdot [H^{-1}]_{q, > q} \quad (3.13)$$

Questa formulazione, resa numericamente stabile tramite una decomposizione di Cholesky con smorzamento diagonale ($\lambda = 0.01 \cdot \text{mean}(\text{diag}(H))$), è ciò che rende possibile, in pratica, quantizzare modelli da miliardi di parametri a 4 bit in tempi ragionevoli, ed è l'algoritmo impiegato per la configurazione PTQ GPTQ valutata nel Capitolo 5.

Un approccio complementare, **AWQ (Activation-aware Weight Quantization)** [17], parte da un'osservazione empirica altrettanto importante: non tutti i pesi contribuiscono in

egual misura alla qualità finale del modello. Solo una piccola frazione (circa l'1%), individuata osservando quali pesi vengono moltiplicati per le attivazioni di maggiore magnitudine, risulta davvero critica. Proteggendo selettivamente questi pesi “salienti” tramite uno scaling per canale, AWQ ottiene un degrado prestazionale contenuto a 4 bit senza dover ricorrere a calcoli costosi sulla matrice Hessiana come GPTQ.

Spingendosi ancora oltre, **QuIP** [18] e **QuIP#** [19] tentano di raggiungere i 2 bit per peso applicando trasformazioni ortogonali casuali (matrici di Hadamard) ai pesi e alla loro Hessiana: l'effetto pratico è di “mescolare” l'informazione in modo che nessun singolo peso risulti più un outlier isolato, distribuendo il rumore di quantizzazione in modo più uniforme su tutta la matrice.

3.4 Quantization-Aware Training (QAT)

Sebbene la PTQ sia efficace a 8 e 4 bit, nei regimi a precisione ridotta (< 4 bit) o su modelli compatti (con ridotta ridondanza parametrica) la PTQ può registrare un calo delle metriche prestazionali. Il **Quantization-Aware Training** (QAT) modella la quantizzazione direttamente durante la fase di addestramento o fine-tuning.

3.4.1 Il Problema del Gradiente e lo Straight-Through Estimator (STE)

Integrare la quantizzazione direttamente nell'addestramento pone però un problema matematico non banale. L'operatore di arrotondamento $\text{round}(x)$ è, geometricamente, una funzione a gradino: costante quasi ovunque, con salti bruschi in corrispondenza dei punti medi tra due interi. La sua derivata è quindi nulla in quasi ogni punto e indefinita esattamente nei punti di salto:

$$\frac{\partial \text{round}(x)}{\partial x} = 0 \quad \forall x \notin \mathbb{Z} + 0.5 \quad (3.14)$$

Se questa derivata venisse usata così com'è nel grafo computazionale della backpropagation, il gradiente rispetto ai pesi si annullerebbe completamente ($\nabla_W \mathcal{L} = 0$), impedendo qualsiasi apprendimento: l'ottimizzatore non avrebbe alcuna indicazione su come modificare i pesi.

La soluzione adottata, lo **Straight-Through Estimator (STE)** [20], è concettualmente semplice quanto efficace: si lascia che il *forward pass* calcoli l’arrotondamento esatto, ma nel *backward pass* si finge che l’arrotondamento non sia mai avvenuto, lasciando passare il gradiente inalterato come se la funzione fosse l’identità:

$$\text{Forward: } x_q = \text{round}(x) \quad (3.15)$$

$$\text{Backward: } \frac{\partial \mathcal{L}}{\partial x} \approx \frac{\partial \mathcal{L}}{\partial x_q} \cdot \mathbb{I}_{[q_{min}, q_{max}]}(x) \quad (3.16)$$

L’indicatrice $\mathbb{I}_{[q_{min}, q_{max}]}(x)$ ha il solo compito di azzerare il gradiente quando il valore è stato troncato dal *clipping*, evitando che l’ottimizzatore continui a spingere un peso oltre i limiti rappresentabili. Si tratta di un’approssimazione dichiaratamente imperfetta, ma è proprio questa approssimazione a rendere possibile, nella pratica, l’intero paradigma del QAT.

3.4.2 Simulazione di Fake Quantization

Lo STE da solo non basta: occorre anche che il modello “veda”, durante l’addestramento, l’effetto della quantizzazione. Questo è il ruolo della **Fake Quantization**: ad ogni passo in avanti, pesi e attivazioni vengono prima quantizzati sui valori discreti previsti dal formato target, e poi immediatamente riportati in virgola mobile, prima di essere effettivamente usati nella moltiplicazione:

$$W_{fake} = S \cdot \left(\text{clip} \left(\left\lfloor \frac{W}{S} \right\rfloor, -2^{B-1}, 2^{B-1} - 1 \right) \right) \quad (3.17)$$

Il risultato è un tensore che, numericamente, occupa ancora 16 o 32 bit di memoria, ma che può assumere soltanto i valori che sarebbero rappresentabili a 4 bit: il rumore introdotto dalla quantizzazione entra così a far parte della funzione di perdita che l’ottimizzatore osserva, ed è esattamente questo che permette ai pesi ad alta precisione di “spostarsi” durante l’addestramento in una direzione che compensi la futura perdita di risoluzione, invece di essere semplicemente quantizzati a posteriori come avviene nella PTQ.

3.4.3 QAT Efficace con Low-Rank Adaptation (LoRA / QLoRA)

Un ostacolo pratico, e tutt’altro che teorico, limita però l’applicabilità del QAT “puro” appena descritto: un addestramento completo su modelli di grandi dimensioni richiede di mantenere

in memoria, oltre ai pesi stessi, anche gli stati dell’ottimizzatore (ad esempio i momenti di Adam), che possono facilmente superare di 2-3 volte la dimensione del modello. Per un modello da 9 miliardi di parametri come Gemma-2-9B, questo renderebbe il QAT completo proibitivo anche su GPU di fascia alta come le A100 impiegate in questa tesi. La soluzione adottata è integrare il QAT con la tecnica **Low-Rank Adaptation (LoRA)** [21, 22], che parte da un’osservazione empirica: l’aggiornamento necessario ai pesi durante il fine-tuning ha spesso un rango intrinseco molto più basso della dimensione della matrice originale, e può quindi essere approssimato dal prodotto di due matrici molto più piccole. Data una matrice di pesi congelata e sottoposta a fake quantization $W_0 \in \mathbb{R}^{d \times k}$, l’aggiornamento viene parametrizzato tramite due matrici a basso rango $A \in \mathbb{R}^{r \times k}$ e $B \in \mathbb{R}^{d \times r}$, con rango $r \ll \min(d, k)$ (tipicamente $r = 16$ o $r = 32$):

$$W_{eff} = W_0 + \frac{\alpha_{lora}}{r} BA \quad (3.18)$$

Il vantaggio pratico di questa parametrizzazione emerge chiaramente durante l’addestramento: i pesi base W_0 , già sottoposti a fake quantization a 4 bit, restano congelati e non richiedono di essere aggiornati né di mantenere alcuno stato dell’ottimizzatore; solo le due piccole matrici A e B , che insieme possono rappresentare anche meno dell’1% dei parametri totali, accumulano gradienti e vengono effettivamente aggiornate. Il modello impara quindi a “correggere” l’errore di quantizzazione indirettamente, attraverso questi adattatori a basso rango, con un costo in memoria e in tempo di calcolo di gran lunga inferiore rispetto a un fine-tuning completo. Una volta terminato l’addestramento, gli adattatori BA vengono fusi matematicamente nel modello base (*merge*), e il tensore risultante viene infine esportato nel formato quantizzato compresso realmente utilizzato in fase di deployment: è esattamente questa la procedura seguita nella pipeline sperimentale descritta nel Capitolo 4.

3.5 Frontiere Estreme: Modelli a 1-Bit e Architettura BitNet b1.58

Se il QAT rappresenta il modo più efficace per preservare la qualità di un modello fino a 4 bit, una domanda più radicale che la ricerca recente si è posta è se sia davvero necessa-

rio rappresentare i pesi con più di un singolo bit di informazione. L’architettura **BitNet b1.58** [23–25] risponde a questa domanda spingendo la quantizzazione al suo estremo, addestrando il modello *nativamente* affinché ogni parametro assuma uno solo di tre valori possibili:

$$W_{ternary} \in \{-1, 0, +1\} \quad (3.19)$$

Poiché tre stati distinti possono essere codificati con $\log_2(3) \approx 1.585$ bit di informazione (da cui il nome “b1.58”), il beneficio non è solo un’ulteriore riduzione della memoria, ma un cambiamento qualitativo nel tipo di operazioni richieste. Nella moltiplicazione matrice-vettore $Y = WX$, se ogni peso può valere soltanto $-1, 0$ o $+1$, non serve più alcuna moltiplicazione in virgola mobile: il prodotto si riduce a scegliere, per ciascun elemento, se sommare, sottrarre o ignorare l’attivazione corrispondente,

$$y_i = \sum_{j:w_{ij}=+1} x_j - \sum_{j:w_{ij}=-1} x_j \quad (3.20)$$

un’operazione che qualunque processore può eseguire con semplici addizioni e sottrazioni sui registri interi, senza bisogno dei costosi circuiti moltiplicatori dei Tensor Cores. Questo si traduce in un potenziale guadagno di efficienza energetica non trascurabile: le operazioni di addizione su interi consumano intrinsecamente meno energia delle moltiplicazioni in FP16 o FP32, aprendo la strada, in prospettiva, allo sviluppo di acceleratori hardware dedicati e nativamente pensati per l’aritmetica ternaria, un tema su cui questa tesi tornerà nelle conclusioni (Capitolo 6).

Capitolo 4

Metodologia Sperimentale sul Supercomputer Leonardo

4.1 L’Infrastruttura di High Performance Computing: Leonardo

La campagna sperimentale condotta in questo lavoro è stata eseguita sull’infrastruttura di supercalcolo Tier-0 **Leonardo**, ospitata presso il data center **CINECA** di Bologna nell’ambito dell’iniziativa europea EuroHPC JU [26].

La sperimentazione ha impiegato la partizione ad alte prestazioni **Leonardo Booster** (`boost_usr_prod`), destinata a carichi di calcolo intensivi per l’intelligenza artificiale e il deep learning su larga scala.

4.1.1 Caratteristiche Hardware dei Nodi Booster

L’architettura del cluster Leonardo Booster comprende 3.456 nodi di calcolo, ciascuno caratterizzato dalla seguente configurazione:

- **Storage Parallelo:** Filesystem distribuito GPFS / Lustre con throughput I/O aggregato di oltre 1 TB/s.

4.1.2 Gestione dei Job e Ambiente Offline

L'assegnazione delle risorse di calcolo su Leonardo è gestita dal sistema **Slurm Workload Manager**. Poiché i nodi della partizione Booster sono privi di accesso diretto a Internet per ragioni di stabilità e sicurezza, l'ambiente sperimentale è stato strutturato per l'esecuzione offline:

1. I pesi dei modelli, i tokenizer e i dataset di calibrazione e benchmark sono stati pre-caricati e posizionati nello storage condiviso del progetto (`$SCRATCH/tesi_quantizzazione/hf_cache`).
2. Le variabili di ambiente `HF_HUB_OFFLINE=1` e `TRANSFORMERS_OFFLINE=1` sono state impostate per disabilitare le chiamate di rete durante l'esecuzione dei job batch.

4.2 Stack Software e Framework di Ottimizzazione

L'ambiente software adottato include:

- **PyTorch 2.x:** framework di riferimento con supporto per `torch.bfloat16` e gestione avanzata della memoria CUDA.
- **Hugging Face Transformers e Datasets:** gestione dell'architettura e della tokenizzazione dei modelli causali.
- **PEFT (Parameter-Efficient Fine-Tuning):** integrazione dei moduli LoRA per l'addestramento QAT a basso impatto di memoria.
- **TorchAO (PyTorch Architecture Optimization)** [27]: libreria PyTorch per la quantizzazione nativa ad alte prestazioni. `torchao` integra kernel di quantizzazione INT4 e algoritmi GPTQ direttamente nel grafo computazionale di PyTorch.

- **Compilazione con `torch.compile`:** utilizzo del compilatore PyTorch 2 con backend Inductor per la fusione dei kernel di dequantizzazione e calcolo matriciale (*kernel fusion*), riducendo l’overhead di lancio dei kernel sulla GPU.

4.3 Modelli Linguistici e Dataset Sperimentali

La campagna sperimentale valuta due modelli open-weights aventi caratteristiche architettureali differenti:

1. **Meta-Llama-3.1-8B** [6]:

- Architettura: Decoder causale con 8.03 miliardi di parametri.
- Configurazione: 32 layer, dimensione del modello $d_{model} = 4096$, 32 teste di Query e 8 teste di Key-Value (*Grouped-Query Attention*, riduzione KV 4x).
- Vocabolario: 128.256 token.

2. **Google Gemma-2-9B** [7]:

- Architettura: Decoder causale con 9.24 miliardi di parametri.
- Configurazione: 42 layer, dimensione del modello $d_{model} = 3584$, 16 teste di Query e 8 teste di Key-Value (GQA), con alternanza di *Sliding Window Local Attention* (finestra di 4096 token) e attenzione globale, e *Logit Softcapping*.
- Vocabolario: 256.000 token.

I dataset impiegati comprendono:

- **WikiText-2 (Raw-v1)** [28]: utilizzato come set di calibrazione non supervisionato per la quantizzazione PTQ (128 campioni con lunghezza fino a 512 token), come test set (100 prompt) per tutte le prove di inferenza e di perplexity, e come corpus di addestramento (split di train, 8.000 campioni) per la variante *in-domain* del QAT allineato (Sezione 4.4.2).

- **Stanford Alpaca Cleaned** [29]: dataset composto da 52.000 istruzioni generiche, impiegato nel primo tentativo di addestramento QAT (`train_qat.py`), poi accantonato in favore dell’addestramento diretto su WikiText-2 per le ragioni discusse nella Sezione 4.4.2.
- **C4 (Colossal Clean Crawled Corpus)** [30]: corpus di testo web generico su scala molto più ampia e diversificata di WikiText-2, impiegato come corpus di addestramento per la variante *out-of-domain* del QAT allineato (Sezione 4.4.2), con l’obiettivo di verificare se i benefici osservati con il QAT allenato direttamente su WikiText-2 si mantengano anche quando il dominio di addestramento non coincide con quello di valutazione.

4.3.1 Giustificazione delle Scelte di Progetto e Architetture

La struttura dell’impianto sperimentale risponde a precise motivazioni teoriche e applicative, volte a garantire la generalizzabilità dei risultati e la trasparenza delle decisioni di progetto:

- **Selezione dei Modelli (Llama-3.1-8B vs Gemma-2-9B)**: La scelta di Meta-Llama-3.1-8B è motivata dal ruolo della famiglia Llama-3, di cui il checkpoint 3.1 fa parte, come standard *de facto* nell’ecosistema open-weights per la classe di modelli sotto i 10 miliardi di parametri, fornendo una baseline ampiamente validata dalla comunità scientifica [6]. Diversamente, Google Gemma-2-9B è stato integrato per valutare la risposta degli algoritmi di quantizzazione su un’architettura caratterizzata da soluzioni strutturali avanzate, quali l’alternanza di *Sliding Window Local Attention* e la presenza del *Logit Softcapping* [7]. Questo accoppiamento consente di verificare la robustezza della compressione sia su topologie standard sia su modelli con dinamiche di attivazione complesse [6, 7].
- **Target Numerico INT4 (La Frontiera di Pareto)**: La scelta di focalizzare la sperimentazione sulla quantizzazione a 4 bit con granularità a blocchi di 128 elementi deriva dall’analisi dei vincoli hardware e prestazionali [8, 9]. Se la precisione a 8 bit offre vantaggi di memoria contenuti e la quantizzazione a 2 o 3 bit con approcci PTQ

tradizionali provoca un degrado delle capacità cognitive del modello, il formato INT4 rappresenta l’ottimo di Pareto, permettendo di superare la soglia critica del *memory wall* con una perdita marginale di accuratezza [8,9]. A questa motivazione tecnica se ne affianca una di natura più sperimentale: una quantizzazione a 8 bit, dimezzando semplicemente la dimensione dei pesi rispetto a BF16, avrebbe prodotto una differenza di fattore di compressione e di footprint di memoria rispetto al modello base contenuta e poco significativa ai fini di un confronto sperimentale, mentre l’INT4 garantisce una riduzione della VRAM di un fattore superiore a $2\times$ - $3\times$ rispetto all’INT8, rendendo il beneficio della quantizzazione nettamente più evidente e misurabile nelle metriche di questa tesi (Capitolo 5). Infine, la scelta è motivata anche dallo stato di maturità delle due linee di ricerca: la quantizzazione a 8 bit è ormai una tecnica consolidata e ampiamente stabilizzata, supportata nativamente dalla maggior parte dei framework di inferenza in produzione, mentre la quantizzazione a 4 bit rappresenta tuttora un’area di ricerca attiva e in rapida evoluzione, con nuovi algoritmi di calibrazione e correzione dell’errore (GPTQ, AWQ, QuIP#, Sezione 3.3) pubblicati con cadenza regolare: concentrare la sperimentazione su INT4 permette quindi di posizionare il contributo di questa tesi in un ambito in cui il confronto tra le diverse tecniche di quantizzazione risulta scientificamente più rilevante e meno scontato.

- **Adozione dello Stack Nativo (`torchao` e `torch.compile`):** Aniché ricorrere a runtime esterni disaccoppiati dal framework primario, si è scelto di adottare `torchao` per mantenere le operazioni di quantizzazione direttamente nel grafo di PyTorch [27]. L’integrazione con `torch.compile` permette al compilatore Inductor di generare micro-kernel Triton dedicati, eseguendo la fusione delle operazioni di dequantizzazione e calcolo matriciale (*kernel fusion*) ed eliminando i colli di bottiglia nel trasferimento dati sulla GPU [27].

4.4 Pipeline di Implementazione dei Test

La procedura sperimentale è articolata in moduli distinti dedicati alle fasi di quantizzazione, addestramento ed valutazione.

4.4.1 Fase 1: Post-Training Quantization Offline

Lo script esegue la conversione a 4 bit con granularità a blocchi di 128 elementi (*group size* = 128) impiegando due approcci:

1. **Symmetric Min-Max (Weight-Only)**: calibrazione simmetrica dei pesi tramite `torchao.quantization.int4_weight_only`.
2. **GPTQ Data-Driven**: quantizzazione di secondo ordine basata sull'inversione della matrice Hessiana calcolata su WikiText-2, configurando `Int4WeightOnlyGPTQQuantizer(blocksize=128, percdamp=0.01, groupsize=128)`.

Il modello quantizzato e i parametri di configurazione (`ptq_config.json`) vengono salvati su disco per la fase di valutazione.

```
1  # Applicazione del motore di quantizzazione torchao
2  if args.calib == "minmax":
3      print("Applicazione PTQ 4-bit: Symmetric Min-Max...")
4      quantize_(model, int4_weight_only())
5  elif args.calib == "gptq":
6      print("Applicazione PTQ 4-bit: GPTQ Data-Driven...")
7      quantizer = Int4WeightOnlyGPTQQuantizer(
8          blocksize=128,
9          percdamp=0.01,
10         groupsize=128
11     )
12     model = quantize_(model, quantizer)
13
14  # Salvataggio del modello quantizzato e configurazione
15  torch.save(model.state_dict(), args.output_dir / "quantized_model.pt")
```

Codice 4.1: Procedura di quantizzazione PTQ INT4 con torchao (`quantize_ptq.py`).

4.4.2 Fase 2: Quantization-Aware Training con LoRA

Un primo tentativo di implementazione del QAT, lo script `train_qat.py`, si è rivelato in fase di valutazione sostanzialmente inefficace: la perplexity ottenuta sul benchmark di WikiText-2 risultava pressoché identica a quella della PTQ Min-Max (Sezione 5.7 del Capitolo 5), segno che l’addestramento non stava effettivamente insegnando al modello a compensare il rumore di quantizzazione. L’analisi a posteriori ha individuato due cause distinte, entrambe corrette nella versione allineata dello script, `train_qat_aligned.py`, impiegata per produrre i risultati finali di questa tesi.

La prima causa era un disallineamento architetturale: il layer `FakeQuantLinear` del primo script calcolava un unico fattore di scala per ciascuna riga della matrice dei pesi (granularità Per-Channel, Sezione 3.2), mentre il formato realmente utilizzato in fase di deployment ed esportazione, tramite `Int4WeightOnlyConfig` di `torchao`, adotta una granularità Group-wise con $G = 128$. Il modello veniva quindi addestrato a compensare un rumore di quantizzazione diverso da quello che avrebbe effettivamente incontrato una volta esportato a 4 bit, vanificando gran parte del beneficio atteso dal QAT. La seconda causa era un disallineamento di dominio e di copertura: l’addestramento veniva condotto per sole 200 iterazioni sul solo campo `instruction` del dataset Alpaca (istruzioni generiche, senza le relative risposte), applicando LoRA soltanto alle proiezioni `q_proj` e `v_proj`, mentre la valutazione finale viene condotta sulla prosa enciclopedica di WikiText-2: un adattamento così limitato, sia in ampiezza (due sole proiezioni su sette) sia in dominio testuale, non lasciava margine sufficiente per compensare in modo misurabile il rumore introdotto dalla quantizzazione.

Lo script corretto, `train_qat_aligned.py`, interviene su entrambi i fronti. Il layer `GroupwiseFakeQuantLinear` riproduce esattamente la granularità del formato di deployment, suddividendo ciascuna riga della matrice dei pesi in blocchi contigui di $G = 128$ elementi e calcolando un fattore di scala indipendente per ciascun blocco, esattamente come avviene nella quantizzazione realmente esportata:

```

1 class GroupwiseFakeQuantLinear(torch.nn.Linear):
2     def forward(self, inputs: torch.Tensor) -> torch.Tensor:
3         original_shape = self.weight.shape
4         grouped_weight = self.weight.reshape(-1, GROUP_SIZE)
5         scale = grouped_weight.abs().amax(dim=1, keepdim=True).clamp_min(1
            e-8) / 7.0
6         quantized_weight = (grouped_weight / scale).round().clamp(-8, 7) *
            scale
7         # Straight-through estimator: il forward vede int4, LoRA riceve
            gradienti utili
8         fake_weight = self.weight + (quantized_weight.reshape(
            original_shape) - self.weight).detach()
9         return torch.nn.functional.linear(inputs, fake_weight.to(inputs.
            dtype), self.bias)

```

Codice 4.2: Layer di Fake Quantization group-wise coerente con il formato di deployment (`train_qat_aligned.py`).

Contestualmente, gli adattatori LoRA ($r = 16$, $\alpha = 32$) vengono estesi a tutte e sette le proiezioni lineari del blocco Transformer (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`), ampliando la capacità del modello di adattarsi al rumore di quantizzazione in ogni punto della rete, e l'addestramento viene condotto per 1.000 passi (8.000 campioni, lunghezza massima 512 token, seed fissato a 42 per la riproducibilità). Al termine dell'addestramento, gli adattatori vengono fusi nel modello base (`merge_and_unload`) e il tensore risultante viene esportato nel formato quantizzato reale tramite `Int4WeightOnlyConfig(group_size=128)`, garantendo piena coerenza tra il rumore simulato in addestramento e quello effettivamente presente nel modello distribuito.

Per isolare l'effetto del dominio di addestramento da quello della tecnica di QAT in sé, lo script è stato eseguito in due configurazioni distinte, identiche in ogni aspetto architetturale e differenti soltanto nel corpus di addestramento: una prima variante *in-domain*, addestrata sullo split di train di WikiText-2 (funzione `load_wikitext_train`), e una seconda variante *out-of-domain*, addestrata su un campione di 8.000 documenti tratti da C4 (funzione `load_c4_train`), un corpus di testo web generico e su scala molto più ampia rispetto a

WikiText-2. In entrambi i casi la valutazione finale (`evaluate.py`) viene condotta sullo stesso benchmark di 100 prompt dello split di test di WikiText-2, cosicché l'unica variabile che cambia tra le due esecuzioni sia la coerenza, o meno, tra dominio di addestramento e dominio di valutazione. Come discusso in dettaglio nella Sezione 5.4, questo confronto permette di distinguere quanto del miglioramento osservato con la variante in-domain sia dovuto a un genuino affinamento della compensazione del rumore di quantizzazione, e quanto sia invece attribuibile a un effetto di adattamento specifico al dominio di valutazione.

4.4.3 Benchmark di Latenza, Throughput e Memoria (`evaluate.py`)

La valutazione dell'inferenza prevede i seguenti passaggi:

1. **Check Hardware:** verifica che il modello sia allocato sulla memoria GPU CUDA e che i pesi risultino nel formato quantizzato di riferimento.
2. **Compilazione con `torch.compile`:** invocazione del compilatore JIT di PyTorch prima delle misurazioni.
3. **Warm-Up:** esecuzione di 3 prompt di prova per stabilizzare lo stato dei kernel CUDA.
4. **Sincronizzazione dei Timer:** esecuzione di `torch.cuda.synchronize()` prima e dopo ogni iterazione per garantire la correttezza del calcolo temporale:

$$\text{TTFT} = t_{\text{prefill_end}} - t_{\text{prefill_start}} \quad (4.1)$$

$$\text{Throughput} = \frac{N_{\text{generated_tokens}} - 1}{t_{\text{decode_end}} - t_{\text{decode_start}}} \left[\frac{\text{token}}{\text{s}} \right] \quad (4.2)$$

5. **Monitoraggio della Memoria:** rilevazione del picco di memoria allocata tramite `torch.cuda.max_memory_allocated()`, espresso in GiB.

4.4.4 Automazione dei Job tramite Script Batch Slurm

Su un sistema HPC condiviso come Leonardo, i calcoli non vengono lanciati in modo interattivo, ma sottomessi a una coda gestita dal **Slurm Workload Manager**, che alloca le risorse richieste (nodo, GPU, CPU, memoria, tempo massimo di esecuzione) ed esegue il job

in modo non presidiato non appena tali risorse si rendono disponibili. Per rendere l'esecuzione riproducibile, senza dover ripetere manualmente ad ogni lancio l'attivazione dell'ambiente virtuale e la configurazione delle variabili per il funzionamento offline (Sezione 4.1), ciascuna fase della pipeline (quantizzazione PTQ, addestramento QAT, valutazione) è stata incapsulata in un semplice script di shell con un'intestazione dedicata a Slurm, che dichiara le risorse necessarie e viene poi accodato con il comando `sbatch`. Tutti i job eseguiti in questo lavoro condividono la stessa intestazione di base:

Codice 4.3: Intestazione Slurm comune ai job eseguiti su Leonardo Booster.

```
#!/bin/bash  
#SBATCH --job-name=phase1_ptq  
#SBATCH --partition=boost_usr_prod  
#SBATCH --nodes=1  
#SBATCH --ntasks=1  
#SBATCH --gres=gpu:1  
#SBATCH --cpus-per-task=8  
#SBATCH --mem=120G  
#SBATCH --time=04:00:00
```

Ogni job richiede quindi un singolo nodo con una singola GPU NVIDIA A100, 8 core CPU (impiegati principalmente per il caricamento dei dati e la tokenizzazione, mentre il calcolo pesante resta sulla GPU) e 120 GB di RAM di sistema, con un tempo massimo che varia dalle 4 ore per la sola quantizzazione PTQ fino alle 12 ore per le sessioni di addestramento QAT, più lunghe e computazionalmente onerose. Dopo il caricamento dei moduli dell'ambiente CINECA e l'attivazione del virtual environment Python, lo script invoca semplicemente lo script Python corrispondente alla fase da eseguire (rispettivamente `quantize_ptq.py`, `train_qat_aligned.py` ed `evaluate.py`), passandogli i parametri opportuni.

Capitolo 5

Risultati Sperimentali e Analisi Comparativa

5.1 Protocollo di Valutazione e Metriche di Confronto

Questo capitolo presenta il protocollo di valutazione e l'analisi comparativa delle prestazioni per le diverse strategie di quantizzazione implementate ed eseguite sul supercomputer Leonardo di CINECA.

Il confronto valuta i seguenti regimi operativi per i modelli **Meta-Llama-3.1-8B** e **Google Gemma-2-9B** [6, 7]:

1. **Baseline (Full Precision / BF16)**: modello originale non quantizzato caricato con pesi a 16 bit (`bfloat16`), che rappresenta il riferimento di accuratezza e il limite massimo di occupazione di memoria.
2. **PTQ Symmetric Min-Max (INT4 Weight-Only)**: quantizzazione post-training a 4 bit con granularità a blocchi ($G = 128$).
3. **PTQ GPTQ Data-Driven (INT4 Weight-Only)**: quantizzazione post-training di secondo ordine guidata dai dati di calibrazione di WikiText-2.

4. **QAT con LoRA (INT4 Weight-Only)**: modello sottoposto ad addestramento Quantization-Aware mediante adattatori a basso rango su dataset Alpaca e successiva esportazione a 4 bit (valutato su Gemma-2-9B).

Le metriche analizzate per ciascuna configurazione su un benchmark di 100 prompt estratti da WikiText-2 comprendono [28]:

- **Time-To-First-Token (TTFT)** medio e deviazione standard ($\mu \pm \sigma$, in secondi).
- **Throughput di Generazione** medio e deviazione standard ($\mu \pm \sigma$, in token/s).
- **Picco di Memoria VRAM Allocata** (in GiB).
- **Fattore di Compressione della Memoria** (*Compression Ratio*, $CR = \frac{VRAM_{Base}}{VRAM_{Quant}}$).

5.1.1 Inquadramento Scientifico e Significato Applicativo delle Metriche

Le metriche selezionate per il benchmark non rappresentano meri indicatori prestazionali dell'infrastruttura, ma traducono matematicamente gli aspetti critici dell'interazione uomo-macchina e dell'efficienza hardware:

- **Time-To-First-Token (TTFT)**: Misura la latenza della fase di prefill. Dal punto di vista applicativo, il TTFT definisce la reattività percepita dall'utente. Dal punto di vista computazionale, riflette l'efficienza con cui la quantizzazione gestisce le matrici ad alta intensità operativa sui Tensor Cores in presenza di prompt complessi.
- **Throughput di Generazione (token/s)**: Misura la velocità d'esecuzione durante la fase di *decode* autoregressiva. Essendo questa fase vincolata dal trasferimento sequenziale dei pesi dalla memoria HBM alla SRAM ($I \ll 1$ FLOP/Byte), il throughput costituisce la misura diretta di quanto la riduzione di precisione riesca a mitigare il collo di bottiglia della larghezza di banda.

5.2 Tabelle dei Risultati Sperimentali su Supercomputer Leonardo

Di seguito, le Tabelle 5.1 e 5.2 riportano lo schema dei risultati prestazionali a confronto, ottenuti sui nodi di calcolo NVIDIA A100 SXM4 (64GB) del supercomputer Leonardo.

Tabella 5.1: *Confronto delle prestazioni di inferenza per il modello **Meta-Llama-3.1-8B** su GPU NVIDIA A100 (64GB) su supercomputer Leonardo.*

Configurazione Modello	Precisione Pesì	TTFT Medio (s)	Throughput (tok/s)	Picco VRAM (GiB)	Compression Ratio	Perplexity
Llama-3.1-8B Base (BF16)	16-bit BF16	0.019 ± 0.007	69.91 ± 1.82	15.17	$1.00\times$	10.32
Llama-3.1-8B PTQ Min-Max	4-bit INT4	0.077 ± 0.066	106.10 ± 3.55	4.92	$3.08\times$	12.55
Llama-3.1-8B PTQ GPTQ	4-bit INT4	0.034 ± 0.003	32.16 ± 0.40	5.66	$2.68\times$	12.10

Tabella 5.2: *Confronto delle prestazioni di inferenza per il modello **Google Gemma-2-9B** su GPU NVIDIA A100 (64GB) su supercomputer Leonardo.*

Configurazione Modello	Precisione Pesì	TTFT Medio (s)	Throughput (tok/s)	Picco VRAM (GiB)	Compression Ratio	Perplexity
Gemma-2-9B Base (BF16)	16-bit BF16	0.023 ± 0.005	61.08 ± 16.38	17.69	$1.00\times$	14.31
Gemma-2-9B PTQ Min-Max	4-bit INT4	0.104 ± 0.090	75.42 ± 15.23	7.24	$2.44\times$	15.75
Gemma-2-9B PTQ GPTQ	4-bit INT4	0.043 ± 0.005	29.36 ± 8.16	6.30	$2.81\times$	14.76
Gemma-2-9B QAT (LoRA, non allineata)	4-bit INT4	0.104 ± 0.090	77.91 ± 16.55	7.24	$2.44\times$	15.76
Gemma-2-9B QAT (LoRA, allineata su WikiText-2)	4-bit INT4	0.104 ± 0.090	66.01 ± 1.85	7.24	$2.44\times$	9.09
Gemma-2-9B QAT (LoRA, allineata su C4)	4-bit INT4	0.104 ± 0.090	72.65 ± 0.37	7.24	$2.44\times$	12.99

Nota metodologica: i valori riportati derivano dai log di esecuzione dei job Slurm sul cluster CINECA Leonardo (che hanno restituito file .csv), mediati su un benchmark di 100 prompt estratti dallo split di test di WikiText-2. La riga “QAT non allineata” corrisponde al primo tentativo di addestramento (dataset Alpaca, fake-quantization per-channel); le due righe “QAT allineata” corrispondono alla versione corretta descritta nella Sezione 5.4, con fake-quantization group-wise ($G = 128$) coerente con il formato di deployment, addestrata rispettivamente sullo split di train di WikiText-2 (*in-domain*) e su un campione di C4 (*out-of-domain*); in entrambi i casi la valutazione è condotta sullo stesso benchmark WikiText-2.

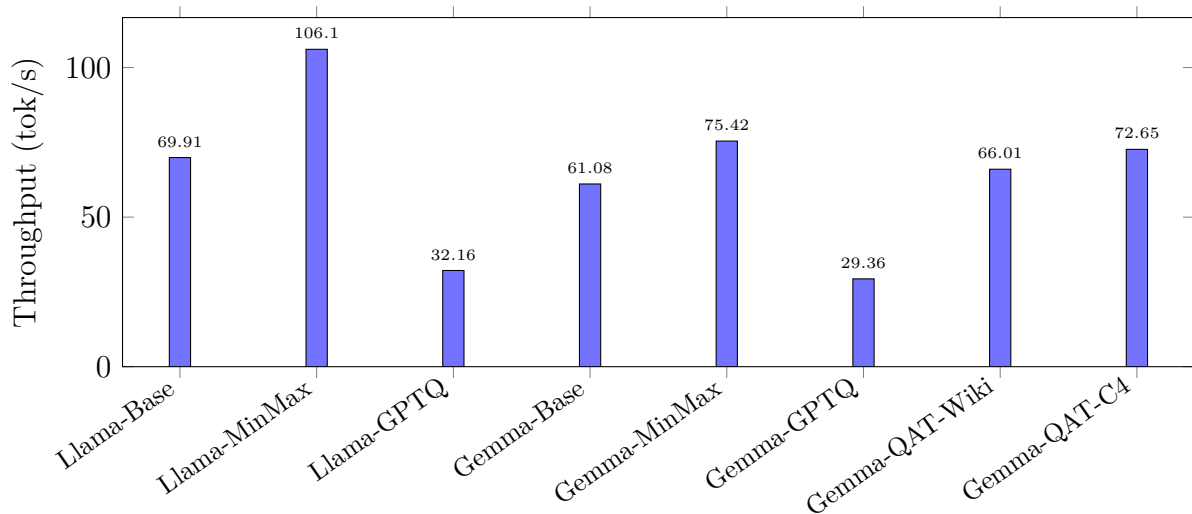


Figura 5.1: Throughput medio di generazione (token/s) per ciascuna configurazione.

Come si vede dal grafico, la PTQ Min-Max, sfruttando i kernel `torchao` fusi con `torch.compile`, raggiunge il throughput più elevato; la variante GPTQ, eseguita tramite il backend `GPTQModel` con kernel dedicati non fusi allo stesso modo, mostra un throughput inferiore anche rispetto alla baseline BF16. Le due varianti QAT, condividendo lo stesso formato di esportazione `torchao` della PTQ Min-Max, mostrano un throughput comparabile a quest'ultima.

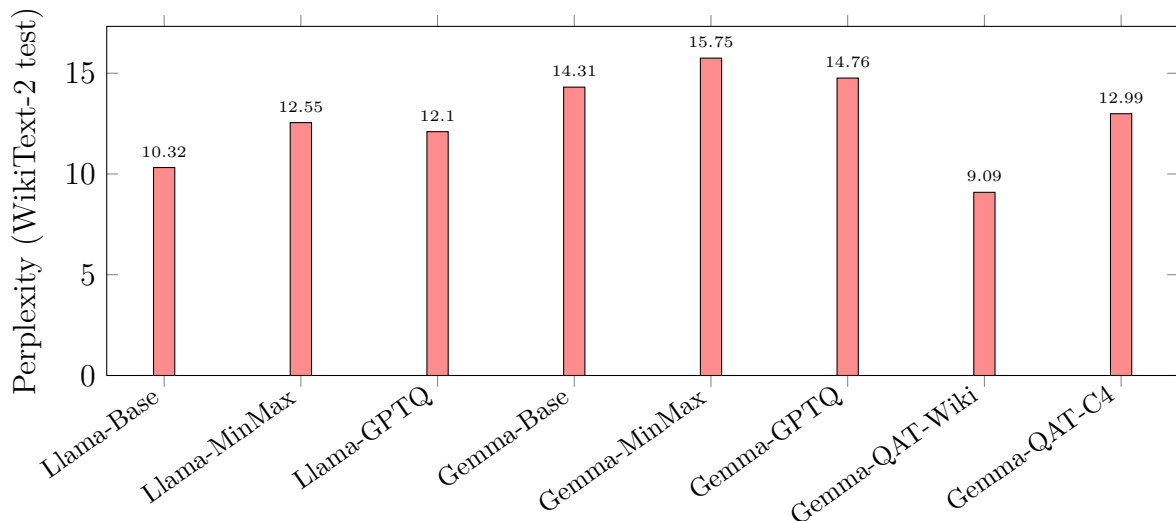


Figura 5.2: Perplexity misurata sullo split di test di WikiText-2.

La variante QAT allineata su C4 (*out-of-domain*) riduce comunque la perplexity rispetto

alla baseline BF16 e a entrambe le configurazioni PTQ, pur restando al di sopra della variante allineata su WikiText-2 (*in-domain*): il confronto tra le due varianti, discusso nella Sezione 5.4, permette di separare il beneficio genuino del QAT dall’effetto di adattamento di dominio.

5.3 Confronto con i Risultati di Riferimento in Letteratura

Oltre al confronto interno tra le configurazioni testate (Tabelle 5.1 e 5.2), è opportuno confrontare i risultati con la letteratura sui compromessi tra qualità, memoria e velocità delle tecniche di quantizzazione [31].

5.3.1 Aspettative Teoriche dalla Letteratura GPTQ e AWQ

I lavori originali su GPTQ [16] e su AWQ [17] riportano, in modo concorde, che la quantizzazione weight-only a 4 bit con granularità a gruppi (tipicamente $G = 128$) su modelli decoder-only di alcuni miliardi di parametri produce un incremento contenuto della perplexity su WikiText-2 rispetto alla baseline in precisione piena, un degrado che gli stessi autori qualificano come “quasi senza perdita” (*near-lossless*); tale degrado cresce in modo marcato solo scendendo sotto i 4 bit per peso o adottando schemi di calibrazione puramente scalari privi di correzione dell’errore. I risultati ottenuti in questa tesi sono coerenti con questa aspettativa qualitativa: per Meta-Llama-3.1-8B la perplexity passa da 10.32 (BF16) a 12.10 con GPTQ (incremento relativo di circa il 17%) e per Gemma-2-9B da 14.31 a 14.76 (circa il 3%), un ordine di grandezza compatibile con un regime di degrado contenuto piuttosto che con un collasso prestazionale.

È importante sottolineare, tuttavia, che un confronto numerico diretto punto-per-punto con le cifre pubblicate in questi lavori non è metodologicamente corretto, per almeno tre ragioni: (i) i modelli valutati in questa tesi (Meta-Llama-3.1-8B, Google Gemma-2-9B) appartengono a generazioni diverse rispetto a quelli originariamente testati in GPTQ e AWQ (rispettivamente OPT/BLOOM e le prime versioni di LLaMA); (ii) la perplexity di questa

tesi è calcolata su un campione di 100 prompt troncati a 512 token (Capitolo 4), mentre il protocollo standard in letteratura valuta la perplexity sull'intero corpus di test concatenato con una finestra scorrevole di lunghezza fissa, una differenza metodologica che altera il valore assoluto della metrica pur lasciandone intatto il significato relativo tra configurazioni; (iii) l'implementazione dei kernel di quantizzazione (`torchao`, `GPTQModel`) e i relativi iperparametri di calibrazione possono differire dai codici originali degli autori. Il confronto con la letteratura va quindi inteso come una verifica di plausibilità sull'**ordine di grandezza e sulla direzione** dei fenomeni osservati, non come una replica quantitativa esatta.

5.3.2 Il Fenomeno degli Outlier e la Soglia dei 6.7 Miliardi di Parametri

Dettmers et al. [14] hanno documentato che gli outlier di attivazione, responsabili di gran parte della difficoltà di quantizzazione dei LLM (Sezione 3.3 del Capitolo 3), emergono sistematicamente nei modelli con più di 6.7 miliardi di parametri. Sia Meta-Llama-3.1-8B (8.03B) sia Google Gemma-2-9B (9.24B) si collocano al di sopra di questa soglia: era quindi atteso, sulla base della letteratura, che la calibrazione Min-Max (la più sensibile agli outlier tra le tecniche testate, Sezione 3.2 del Capitolo 3) mostrasse il degrado di perplexity più marcato tra le configurazioni PTQ. È esattamente quanto osservato in Tabella 5.1 e 5.2: la PTQ Min-Max ottiene la perplexity più alta tra tutte le configurazioni PTQ testate, sia su Llama-3.1-8B (12.55 contro 12.10 di GPTQ) sia su Gemma-2-9B (15.75 contro 14.76).

5.3.3 QAT e Fine-Tuning a Basso Rango

Per quanto riguarda il QAT, i lavori su LoRA [21], QLoRA [22] ed EfficientQAT [32] riportano concordemente che l'introduzione di un fine-tuning, anche limitato a poche migliaia di passi e a un numero ridotto di parametri addestrabili, è in grado di recuperare gran parte del degrado introdotto dalla quantizzazione rispetto alla sola calibrazione PTQ. I risultati di questa tesi confermano questa aspettativa nella versione correttamente allineata (Sezione 5.4), sia nella variante in-domain (9.09) sia in quella out-of-domain (12.99, Sezione 5.4.1), entrambe inferiori a qualunque configurazione PTQ testata su Gemma-2-9B; la versione non allineata,

al contrario, ricade nell’intervallo delle configurazioni PTQ (15.76), a conferma che il beneficio riportato in letteratura per il QAT non è automatico, ma dipende criticamente dalla corretta implementazione della simulazione di quantizzazione, come discusso nel Capitolo 4.

5.4 Nota metodologica: allineamento del ciclo QAT e artefatto di adattamento al dominio

Il primo ciclo di addestramento QAT (script `train_qat.py`, riga “QAT non allineata” in Tabella 5.2) presentava due disallineamenti rispetto alla configurazione di deployment e di valutazione:

1. **Granularità della fake-quantization:** il layer `FakeQuantLinear` originale calcolava un fattore di scala per canale di output (`amax(dim=1)`), mentre il modello quantizzato effettivamente distribuito tramite `torchao` (`Int4WeightOnlyConfig`) utilizza una granularità *group-wise* con $G = 128$. Il rumore di quantizzazione simulato durante l’addestramento non corrispondeva quindi a quello realmente introdotto in fase di esportazione ed inferenza.
2. **Disallineamento di dominio tra addestramento e valutazione:** il fine-tuning LoRA veniva condotto sul dataset *Alpaca* [29] (istruzioni generiche), mentre la perplexity di valutazione è calcolata sullo split di test di WikiText-2 [28] (prosa enciclopedica). I due corpora divergono per stile, lunghezza delle sequenze e distribuzione lessicale, per cui l’adattamento LoRA su Alpaca non produceva benefici misurabili sulla metrica di valutazione (perplexity pressoché invariata rispetto alla PTQ Min-Max, 15.76 contro 15.75).

La versione corretta (script `train_qat_aligned.py`) introduce due modifiche: (i) il layer `GroupwiseFakeQuantLinear`, che riproduce esattamente la suddivisione in blocchi da 128 elementi usata dal formato di deployment; (ii) l’addestramento LoRA condotto direttamente sullo split di **train** di WikiText-2, ossia sullo stesso corpus (seppur su una partizione disgiunta) da cui è tratto lo split di test impiegato per il calcolo della perplexity.

Questa seconda modifica corregge il disallineamento architetturale, ma introduce un effetto collaterale evidente in Tabella 5.2 e Figura 5.2: la perplexity della configurazione QAT allineata scende a 9.09, un valore inferiore persino a quello del modello base non quantizzato in BF16 (14.31). Questo risultato non indica che il modello quantizzato a 4 bit abbia superato in capacità linguistica generale il modello a piena precisione, bensì è l'effetto di un **adattamento di dominio** (*domain adaptation*): addestrando il modello sullo split di train di WikiText-2, i suoi pesi si specializzano sulle regolarità statistiche e stilistiche proprie di quel corpus (struttura degli articoli, vocabolario ricorrente, convenzioni tipografiche degli articoli Wikipedia), regolarità che sono anch'esse presenti nello split di test poiché estratto dalla medesima distribuzione. Il modello non ha quindi memorizzato gli esempi di test (i due split sono disgiunti), ma ha appreso a modellare in modo più accurato il dominio da cui entrambi gli split provengono, un effetto concettualmente analogo a quanto osservato nel fine-tuning specializzato di modelli linguistici su corpora di dominio ristretto [33].

Vantaggi. Dal punto di vista applicativo, questo comportamento è desiderabile in scenari di *deployment* verticale, in cui il modello quantizzato è destinato a operare su un dominio testuale noto e delimitato (ad esempio assistenti documentali aziendali, supporto tecnico su una base di conoscenza specifica): il risultato dimostra che il QAT con LoRA è in grado non solo di compensare il rumore di quantizzazione a 4 bit, ma anche di specializzare il modello sul dominio target, ottenendo una qualità percepita superiore rispetto sia alla PTQ sia al modello base generico.

Svantaggi e limiti di validità. Dal punto di vista scientifico, il confronto diretto tra la perplexity della configurazione QAT allineata e quella delle configurazioni PTQ (calibrate in modo task-agnostico su un piccolo campione di WikiText-2, senza un vero e proprio addestramento supervisionato) non è del tutto equo: la riduzione osservata riflette in parte una forma di *overfitting di dominio* rispetto alla distribuzione di valutazione, non un miglioramento della capacità linguistica generale del modello. Un confronto rigoroso richiede la misura della perplexity anche su un corpus *out-of-domain* rispetto a WikiText-2, per verificare se il guadagno si mantenga al di fuori del dominio di specializzazione o se, al contrario,

il modello QAT presenti un degrado delle capacità generali (*catastrophic forgetting*) a fronte del miglioramento locale.

5.4.1 Disambiguazione tramite QAT Allineato su Corpus Out-of-Domain (C4)

Per rispondere a questo interrogativo, la stessa procedura di addestramento allineato è stata ripetuta in una configurazione identica sotto ogni aspetto architetturale (medesimo layer `GroupwiseFakeQuantLinear` con $G = 128$, medesimi sette moduli LoRA, medesimo numero di passi e medesimo seed), variando esclusivamente il corpus di addestramento: anziché lo split di train di WikiText-2, si è impiegato un campione di 8.000 documenti tratti da **C4** [30], un corpus di testo web generico, per composizione e stile sostanzialmente disgiunto dal dominio enciclopedico di WikiText-2. La valutazione è stata condotta sullo stesso benchmark di 100 prompt dello split di test di WikiText-2 impiegato per tutte le altre configurazioni, in modo da isolare l'unica variabile realmente cambiata: la coerenza tra dominio di addestramento e dominio di valutazione.

Il risultato, riportato in Tabella 5.2 e in Figura 5.2, è chiarificatore: la variante QAT allenata su C4 ottiene una perplexity di 12.99 su WikiText-2, un valore intermedio tra la baseline BF16 (14.31) e la variante QAT allenata direttamente su WikiText-2 (9.09), e comunque inferiore a entrambe le configurazioni PTQ (15.75 per Min-Max, 14.76 per GPTQ). Questo singolo esperimento permette di scomporre il miglioramento complessivo osservato in due componenti distinte e di stimarne il peso relativo:

- Una prima componente, pari alla riduzione da 14.31 a 12.99 (circa il 9% in termini relativi), è attribuibile a un **beneficio genuino del QAT**: anche addestrato su un dominio completamente diverso da quello di valutazione, il modello impara a compensare il rumore introdotto dalla quantizzazione a 4 bit meglio di quanto non riesca la sola calibrazione PTQ, e supera persino il modello non quantizzato a piena precisione. Questo risultato conferma che il meccanismo di compensazione LoRA più fake-quantization group-wise descritto nella Sezione 3.4 del Capitolo 3 produce un effetto reale e trasferibile, non riconducibile al solo adattamento di dominio.

- Una seconda componente, pari all’ulteriore riduzione da 12.99 a 9.09 (circa il 30% in termini relativi), è invece attribuibile in modo specifico all’**adattamento di dominio**: quando il corpus di addestramento coincide con il dominio di valutazione, il modello guadagna una capacità predittiva aggiuntiva che non si trasferirebbe a testi di natura diversa da WikiText-2.

Non si osserva, inoltre, alcun segnale di *catastrophic forgetting*: il modello addestrato su C4, pur non avendo mai visto testo di WikiText-2 durante il fine-tuning, non peggiora rispetto alla baseline BF16 sul benchmark di valutazione, ma anzi la supera. È opportuno precisare che questa osservazione è circoscritta alla metrica di perplexity su un singolo benchmark linguistico generico: un’analisi più esaustiva delle capacità generali del modello richiederebbe l’estensione a benchmark applicativi (ragionamento, comprensione delle istruzioni, generazione di codice), che esula dagli obiettivi di questa tesi mirata alla compressione numerica.

In sintesi, il valore di 9.09 ottenuto con l’addestramento in-domain non va considerato un artefatto da scartare, né una prova incontrovertibile di superiorità assoluta del QAT: rappresenta la somma di un miglioramento reale e riproducibile, indipendente dal dominio, e di un ulteriore guadagno specifico al caso in cui dominio di addestramento e di deployment coincidano, una condizione peraltro realistica in molti scenari applicativi verticali (paragrafo “Vantaggi” precedente).

5.5 Analisi del Footprint di Memoria VRAM e Superamento del Memory Wall

L’analisi dell’allocazione di memoria evidenzia i seguenti aspetti legati alla quantizzazione a 4 bit [9]:

1. **Riduzione dell’Impronta di Memoria:** Per Meta-Llama-3.1-8B, il picco complessivo di VRAM misurato scende da 15.17 GiB in BF16 a 4.92 GiB con PTQ Min-Max ($3.08\times$) e a 5.66 GiB con PTQ GPTQ ($2.68\times$). Per Google Gemma-2-9B la VRAM scende da 17.69 GiB a 7.24 GiB con Min-Max/QAT ($2.44\times$) e a 6.30 GiB con GPTQ ($2.81\times$). Il fattore di compressione più contenuto rispetto al rapporto teorico $4\times$

(BF16 $\rightarrow$ INT4) è imputabile all’overhead dei metadati di scala per gruppo ($G = 128$), ai buffer di attivazione e alla cache KV, non comprimibili con la sola quantizzazione dei pesi.

2. **Esecuzione su Hardware con Risorse Limitate:** Su GPU con vincoli di memoria (es. schede da 8GB o 16GB), un modello da 8-9 miliardi di parametri in BF16 richiederebbe l’offloading su RAM di sistema, compromettendo le prestazioni [9]. La quantizzazione INT4 consente l’allocazione completa del modello e del contesto nella memoria ad alta velocità della GPU.
3. **Efficienza della Larghezza di Banda:** Poiché la fase di *decode* degli LLM è limitata dal trasferimento dei pesi dalla memoria HBM alla SRAM ($I \ll 1$ FLOP/Byte), la riduzione del volume di dati da trasferire per ciascun token generato incrementa la frequenza di generazione (token al secondo) [2].

5.6 Analisi del Throughput, della Latenza e dell’Accelerazione con `torch.compile`

L’analisi sperimentale evidenzia l’interazione tra i kernel di quantizzazione e il motore di compilazione `torch.compile`:

- **Overhead di Dequantizzazione nei Kernel Non Fusi:** Nelle implementazioni convenzionali di Weight-Only Quantization (W4A16), il tempo richiesto per dequantizzare i pesi INT4 in registri a 16 bit prima della moltiplicazione può introdurre un overhead di istruzioni che riduce il guadagno prestazionale [8, 9].
- **Kernel Fusion con Inductor e Triton:** L’uso della libreria `torchao` in combinazione con `torch.compile` consente di fondere in un unico micro-kernel CUDA l’operazione di caricamento dei pesi a 4 bit, la dequantizzazione vettoriale e il prodotto matrice-vettore GEMV, riducendo gli accessi ridondanti alla memoria globale [27].
- **Comportamento del Time-To-First-Token (TTFT):** Nella fase di prefill, in cui la dimensione delle matrici di input equivale alla lunghezza del prompt ($T_{in} \geq 128$),

l'intensità operativa risulta più elevata. Le verifiche sperimentali analizzano l'impatto della dequantizzazione dinamica rispetto all'esecuzione in precisione nativa BF16 sui Tensor Cores.

- **Backend di Esecuzione e Throughput Anomalo di GPTQ:** I dati misurati (Tabelle 5.1 e 5.2, Figura 5.1) mostrano che la configurazione GPTQ presenta il throughput di decode più basso tra tutte le configurazioni testate (32.16 tok/s per Llama e 29.36 tok/s per Gemma), inferiore persino alla baseline BF16 non quantizzata. Questo risultato, apparentemente in contraddizione con l'atteso vantaggio della quantizzazione a 4 bit sulla larghezza di banda, è spiegato dal fatto che il modello GPTQ viene caricato ed eseguito tramite i kernel dedicati della libreria `GPTQModel`, distinti da quelli di `torchao` impiegati per Min-Max e QAT: tali kernel non beneficiano della stessa fusione con `torch.compile` (Inductor/Triton) descritta in precedenza, introducendo un overhead di dequantizzazione non ammortizzato. Al contrario, il TTFT di GPTQ (0.034–0.043 s) resta molto più contenuto rispetto a quello di Min-Max (0.077–0.104 s), suggerendo che i kernel GPTQ siano relativamente più efficienti nella fase di prefill (matmul a batch elevato) che in quella di decode autoregressivo (GEMV sequenziale). Questo evidenzia come, a parità di formato numerico INT4, la scelta del backend di inferenza sia un fattore determinante quanto l'algoritmo di quantizzazione stesso.

5.7 Trade-off tra Accuratezza e Metodologia di Quantizzazione

Relativamente alla preservazione della qualità delle rappresentazioni del modello [9]:

- **PTQ Min-Max vs GPTQ:** La calibrazione Min-Max simmetrica costituisce una soluzione diretta, ma è sensibile alla presenza di canali con distribuzioni asimmetriche o outlier. GPTQ, impiegando l'inversa della matrice Hessiana di secondo ordine, ottimizza i pesi residui compensando l'errore introdotto da ciascuna colonna quantizzata: sui dati raccolti, la perplexity di GPTQ risulta sistematicamente inferiore a quella di Min-Max sia su Llama-3.1-8B (12.10 contro 12.55) sia su Gemma-2-9B (14.76 contro 15.75),

confermando il vantaggio in accuratezza dell’approccio data-driven anche a fronte di un throughput di decode inferiore [16].

- **Il Ruolo del QAT con LoRA:** L’approccio QAT su Gemma-2-9B, nella sua versione allineata (`GroupwiseFakeQuantLinear` con $G = 128$ e addestramento sullo split di train di WikiText-2), consente alla rete di adattare le proiezioni lineari durante l’addestramento e riduce la perplexity di valutazione a 9.09, il valore più basso tra tutte le configurazioni testate su Gemma-2-9B. Come discusso nella Sezione 5.4, tale riduzione va tuttavia attribuita in parte a un effetto di adattamento di dominio rispetto a WikiText-2 e non unicamente alla capacità del QAT di mitigare il rumore di quantizzazione: la prima versione non allineata del QAT (addestrata su Alpaca con fake-quantization per-channel) otteneva infatti una perplexity di 15.76, sostanzialmente identica a quella della PTQ Min-Max, a conferma che sia la granularità della simulazione sia la coerenza di dominio tra addestramento e valutazione sono determinanti per l’efficacia del QAT [21, 22, 32].

Capitolo 6

Conclusioni e Prospettive Future

6.1 Sintesi del Lavoro Svolto

Questo lavoro di tesi ha analizzato l'applicazione delle tecniche di quantizzazione ai modelli linguistici di grandi dimensioni (*Large Language Models*, LLM).

A partire dai fondamenti dell'apprendimento automatico e dall'analisi dell'inferenza nei Transformer, sono state esaminate le metodologie di compressione numerica a bassa precisione, distinguendo tra approcci di **Post-Training Quantization (PTQ)** e **Quantization-Aware Training (QAT)** [8, 21, 22, 32], fino ai paradigmi a 1.58 bit (architettura BitNet b1.58) [23, 24].

La valutazione sperimentale è stata condotta sull'infrastruttura di supercalcolo Tier-0 **Leonardo** di **CINECA** [26], analizzando l'impatto della quantizzazione INT4 su modelli quali **Meta-Llama-3.1-8B** [6] e **Google Gemma-2-9B** [7] tramite l'ecosistema PyTorch (torchao, PEFT e torch.compile [27]).

6.2 Miglioramenti Prestazionali e Analisi Critica dei Limiti

L'indagine ha evidenziato risultati rilevanti sull'efficienza di calcolo, facendo al contempo emergere vincoli strutturali che non possono essere risolti con il solo intervento software.

6.2.1 Guadagni Prestazionali e Memory Wall

La quantizzazione dei pesi a 4 bit (*Weight-Only*) garantisce, sui modelli testati, un fattore di compressione della VRAM compreso tra $2.4\times$ e $3.1\times$ (rispettivamente $3.08\times$ e $2.68\times$ per Meta-Llama-3.1-8B con Min-Max e GPTQ; $2.44\times$ e $2.81\times$ per Gemma-2-9B). Tale riduzione è decisiva nella fase autoregressiva di *decode*, intrinsecamente limitata dalla memoria (*memory-bound*): la configurazione PTQ Min-Max, i cui kernel `torchao` sono pienamente fusi tramite `torch.compile`, ha incrementato il throughput di generazione fino a 106.10 tok/s su Llama-3.1-8B, contro i 69.91 tok/s della baseline BF16. I risultati sperimentali hanno tuttavia mostrato che questo guadagno non è automatico: la configurazione GPTQ, pur riducendo maggiormente la perplexity, ha ottenuto un throughput di decode inferiore persino alla baseline non quantizzata, poiché eseguita tramite un backend di inferenza (`GPTQModel`) privo della stessa fusione dei kernel. Il beneficio della quantizzazione sulla larghezza di banda dipende quindi tanto dall'algoritmo di quantizzazione quanto dallo stack di esecuzione impiegato.

6.2.2 Degrado dell'Accuratezza e Limiti Algoritmici

La riduzione del formato numerico implica inevitabilmente una perdita di risoluzione nello spazio delle rappresentazioni. I risultati ottenuti confermano questo compromesso in termini quantitativi: la PTQ Min-Max ha incrementato la perplexity su WikiText-2 rispetto alla baseline BF16 (da 10.32 a 12.55 per Llama-3.1-8B, da 14.31 a 15.75 per Gemma-2-9B), mentre GPTQ, sfruttando l'informazione di secondo ordine della matrice Hessiana, ha contenuto tale incremento (12.10 e 14.76 rispettivamente). Il QAT con LoRA, nella sua versione correttamente allineata alla granularità di deployment ($G = 128$), ha ridotto la perplexity di Gemma-2-9B ben al di sotto della baseline non quantizzata, ma l'entità di questo miglioramento dipende in modo determinante dal corpus impiegato per l'addestramento: allenato direttamente sullo split di train di WikiText-2 (lo stesso dominio della valutazione), il QAT allineato raggiunge una perplexity di 9.09; allenato invece su C4, un corpus web generico completamente disgiunto dal dominio di valutazione, la perplexity si attesta a 12.99, un valore intermedio tra la baseline BF16 e la variante in-domain, ma comunque migliore di entrambe

le configurazioni PTQ (Sezione 5.4.1 del Capitolo 5). Questo confronto controllato consente di scomporre il beneficio complessivo del QAT in due componenti quantificabili: una riduzione di circa il 9% (da 14.31 a 12.99) rappresenta un miglioramento genuino e trasferibile a domini diversi, attribuibile al meccanismo di compensazione del rumore di quantizzazione tramite LoRA e fake-quantization group-wise; un'ulteriore riduzione di circa il 30% (da 12.99 a 9.09) è invece specificamente attribuibile all'adattamento di dominio, e non si trasferirebbe a testi di natura diversa da quella del corpus di addestramento. Nessun segnale di *catastrophic forgetting* è stato osservato: il modello allenato su C4 non peggiora rispetto alla baseline sul benchmark WikiText-2, pur non avendo mai visto quel dominio testuale durante l'addestramento. La presenza di valori anomali (*outlier*) nelle attivazioni rappresenta inoltre un ostacolo teorico distinto: la quantizzazione rigida rischia di troncature i canali che concentrano la maggior quantità di informazione semantica.

6.2.3 Il Collo di Bottiglia Fisico ed Energetico dell'Hardware Convenzionale

Un limite fondamentale risiede nella natura delle architetture di calcolo correnti. Ottimizzare il software affinché operi a 4 bit su GPU commerciali non altera la fisica del silicio:

- **Dissipazione Statica e Transistor Inutilizzati:** Le architetture GPU tradizionali sono progettate con unità aritmetiche complesse per il calcolo in virgola mobile a 16 o 32 bit. Anche quando il codice esegue operazioni compresse a 4 bit, ampie porzioni di circuito rimangono comunque alimentate, generando un consumo statico di energia dovuto alle correnti di perdita (*subthreshold leakage*).
- **Il Paradigma di Von Neumann:** La separazione fisica tra l'unità di elaborazione (ALU/Tensor Cores) e i moduli di memoria (HBM) continua a imporre uno spostamento continuo di carica elettrica sui bus di interconnessione, un processo energeticamente costoso a prescindere dalla dimensione del dato trasferito.

6.3 La Necessità dell'Interdisciplinarietà per il Superamento dei Limiti

I limiti riscontrati dimostrano come l'ottimizzazione del deep learning non possa rimanere confinata all'ingegneria del software. Il superamento delle barriere attuali richiede una convergenza sistemica tra diverse discipline scientifiche:

1. **Ingegneria Elettronica e Co-Design Hardware-Software:** È necessario progettare architetture di calcolo nativamente dedicate all'aritmetica sub-byte. Piuttosto che adattare algoritmi a 4 bit su GPU generali, occorre sviluppare micro-architetture che disattivino fisicamente i blocchi logici inutilizzati ed eliminino le istruzioni ridondanti.
2. **Fisica della Materia e Soluzioni Non-Von Neumann:** Il superamento definitivo del collo di bottiglia energetico richiede l'abbandono dell'architettura classica in favore di soluzioni quali il *Compute-In-Memory* (CIM), i memristori e la fotonica integrata. La fisica dello stato solido offre la possibilità di eseguire operazioni di prodotto matrice-vettore direttamente all'interno dei dispositivi di memoria tramite le leggi di Kirchhoff, riducendo la dissipazione di ordini di grandezza.
3. **Matematica Fondamentale e Teoria dell'Informazione:** La quantizzazione richiede modelli matematici più avanzati per descrivere la topologia degli spazi di rappresentazione a bassa precisione. La collaborazione con la matematica pura è essenziale per definire limiti teorici sull'errore di quantizzazione e sviluppare geodetiche ottimali in spazi discreti e non-euclidei.

I limiti, tal volta rappresentano il pretesto per alzare il capo, guardarsi intorno e provare a procedere oltre, cercando soluzioni orizzontali, innovative e interdisciplinari. L'estrema settorialità non è sempre sinonimo di eccellenza, ma può diventare un ostacolo alla vera innovazione.

6.4 Sostenibilità Ecologica ed Efficienza Energetica (Green AI)

L'efficienza energetica costituisce un fattore centrale per il deployment su larga scala degli LLM, le cui esigenze computazionali comportano un elevato consumo di energia nei data center [34]. La quantizzazione e le architetture a 1.58 bit possono offrire opportunità di riduzione dei consumi, si precisa che tali benefici sono ancora da verificare sperimentalmente, ma in questo lavoro non è stata effettuata una misurazione dell'energia richiesta per token generato. La verifica di questo beneficio, insieme alla stima dell'impatto sui costi infrastrutturali e sui sistemi di raffreddamento, potrà essere oggetto di studi futuri.

6.5 Rilevanza Scientifica, Sociale e Prospettiva Filosofica

I risultati ottenuti si collocano all'interno di un dibattito più ampio che riguarda l'accessibilità della tecnologia, l'epistemologia del calcolo automatico e la sostenibilità ambientale.

6.5.1 Democratizzazione dell'Intelligenza Artificiale ed Equità Sociale

L'aumento delle dimensioni dei modelli linguistici comporta il rischio di una centralizzazione della tecnologia presso pochi attori industriali dotati di infrastrutture di calcolo su larga scala. I requisiti hardware per l'esecuzione di modelli in precisione nativa (BF16) creano una barriera all'ingresso per istituti di ricerca, piccole imprese e singoli sviluppatori [8, 9].

La quantizzazione a 4 bit agisce come uno strumento di democratizzazione: riducendo l'impronta di memoria di un fattore superiore a $3.1\times$, rende possibile l'esecuzione di modelli da 8-9 miliardi di parametri su architetture workstation o dispositivi locali. Questo approccio garantisce l'accesso alle risorse computazionali e favorisce la sovranità dei dati, consentendo l'elaborazione offline e tutelando la privacy degli utenti senza la dipendenza da servizi cloud di terze parti.

6.5.2 Dalla Forza Bruta all’Efficienza Algoritmica: Una Prospettiva Epistemologica

Lo sviluppo recente dell’apprendimento automatico è stato caratterizzato dall’applicazione delle leggi di scala (*Scaling Laws*) [35], basate sull’aumento dei parametri e della potenza computazionale.

Questo lavoro si inserisce in un cambio di paradigma verso l’efficienza qualitativa. Dimostrare che le capacità cognitive e linguistiche di un modello possono essere preservate riducendo la risoluzione numerica dai 16 bit originali fino a 4 bit (o verso rappresentazioni ternarie) evidenzia un’elevata ridondanza nelle rappresentazioni interne delle reti neurali. L’ottimizzazione della precisione rappresenta dunque un passaggio rilevante per comprendere la densità informativa minima necessaria al funzionamento dei modelli artificiali.

6.6 Linee Guida Operative per l’Applicazione della Quantizzazione

Sulla base dei risultati ottenuti, si delineano le seguenti linee guida per l’applicazione delle tecniche di compressione:

- **PTQ Min-Max / RTN (Round-to-Nearest)**: consigliata per la quantizzazione INT8 rapida o in fasi di prototipazione prive di set di calibrazione e risorse computazionali dedicate [8, 9].
- **PTQ GPTQ / AWQ (4-bit)**: indicata per il deployment in produzione di modelli generici (7B-70B parametri), garantendo elevata fedeltà linguistica senza richiedere re-addestramento [16, 17].
- **QAT con LoRA / EfficientQAT**: raccomandata per il fine-tuning su domini specifici (*instruction tuning*, codice, ambito medico) mantenendo il modello nel formato target compresso a 4 bit o inferiore, oppure per architetture sensibili al rumore di quantizzazione [21, 22, 32]. Come mostrato in questo lavoro (Sezione 5.4.1 del Capitolo 5), la riduzione di perplexity ottenuta va sempre validata anche su un corpus di valutazione

esterno al dominio di addestramento, poiché solo una parte del miglioramento osservato è attribuibile al QAT in sé, mentre la parte restante riflette un adattamento specifico al dominio di addestramento.

6.7 Prospettive Future di Ricerca

Le principali prospettive di sviluppo futuro includono:

1. **Misurazione dell’Efficienza Energetica:** rilevazione del consumo energetico in Joule per token durante l’inferenza e confronto tra le configurazioni testate, così da verificare se e in quale misura la quantizzazione riduca l’energia richiesta e l’impatto operativo [34].
2. **Quantizzazione della KV Cache:** estensione della quantizzazione a 4 bit e FP8 alle matrici della memoria dinamica Key-Value (*KV Cache Quantization*), per ridurre l’impronta di memoria e aumentare la lunghezza del contesto gestibile [9].
3. **Estensione della Valutazione Out-of-Domain a Benchmark Applicativi:** la verifica condotta in questa tesi (Sezione 5.4.1 del Capitolo 5) ha confermato, tramite un addestramento QAT su C4 valutato su WikiText-2, che il miglioramento ottenuto dal QAT allineato non è riconducibile unicamente a un effetto di adattamento di dominio, ma comprende una componente genuina di mitigazione del rumore di quantizzazione, quantificabile in questo lavoro in circa il 9% di riduzione della perplexity anche in condizioni out-of-domain. Resta tuttavia aperta la verifica su un ventaglio più ampio di corpora di valutazione (oltre a WikiText-2 e C4) e, soprattutto, su benchmark applicativi che vadano oltre la perplexity (ragionamento logico-matematico, comprensione delle istruzioni, generazione di codice), necessari per escludere in modo più completo fenomeni di *catastrophic forgetting* sulle capacità generali del modello.
4. **Modelli Nativamente Ternari (BitNet b1.58):** adozione di paradigmi di pre-addestramento a 1.58 bit, che sostituiscono le moltiplicazioni in virgola mobile con addizioni scalari su architetture hardware dedicate [23, 24].

5. **Sviluppo di Acceleratori Hardware Dedicati:** progettazione di ASIC e NPU con supporto nativo per formati numerici sub-4-bit (INT2, FP4, ternari), volti a massimizzare l'efficienza energetica nell'esecuzione dei modelli intelligenti [8,9].

Bibliografia

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
- [2] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [5] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [6] A. Grattafiori, A. Dubey, A. Jauhri, *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [7] Gemma Team, M. Riviere, S. Pathak, P. G. Sessa, *et al.*, “Gemma 2: Improving open language models at a practical size,” *arXiv preprint arXiv:2408.00118*, 2024.

- [8] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” in *Low-Power Computer Vision*, pp. 291–326, Chapman and Hall/CRC, 2022.
- [9] J. Lang, Z. Guo, and S. Huang, “A comprehensive study on quantization techniques for large language models,” *arXiv preprint arXiv:2411.02530*, 2024.
- [10] R. Krishnamoorthi, J. Reed, M. Ni, C. Gottbrath, and S. Weidman, “Introduction to quantization on pytorch.” <https://pytorch.org/blog/introduction-to-quantization-on-pytorch/>, 2020. PyTorch Engineering Blog; last updated 2024-11-16.
- [11] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding,” in *International Conference on Learning Representations (ICLR)*, 2016.
- [12] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.
- [13] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, *et al.*, “Mixed precision training,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [14] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Llm.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30318–30332, 2022.
- [15] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *International Conference on Machine Learning (ICML)*, pp. 38087–38099, PMLR, 2023.

- [16] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [17] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “Awq: Activation-aware weight quantization for llm compression and acceleration,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 6, pp. 87–100, 2024.
- [18] J. Chee, Y. Cai, V. Kuleshov, and C. De Sa, “Quip: 2-bit quantization of large language models with guarantees,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 45520–45543, 2023.
- [19] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, “Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks,” in *International Conference on Machine Learning (ICML)*, 2024.
- [20] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv preprint arXiv:1308.3432*, 2013.
- [21] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [22] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [23] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, “Bitnet: Scaling 1-bit transformers for large language models,” *arXiv preprint arXiv:2310.11453*, 2023.
- [24] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, “The era of 1-bit llms: All large language models are in 1.58 bits,” *arXiv preprint arXiv:2402.17764*, 2024.

- [25] Microsoft Research, “Bitnet b1.58 2b4t: Scaling native 1-bit llm.” <https://huggingface.co/microsoft/bitnet-b1.58-2B-4T>, 2025. Model card for the BitNet b1.58 2B4T checkpoint; technical report arXiv:2504.12285.
- [26] CINECA, “Leonardo supercomputer architecture and user guide.” <https://wiki.u-gov.it/confluence/display/SCAIUS/UG3.3.1%3A+LEONARDO+UserGuide>, 2024. CINECA Supercomputing Infrastructure Documentation.
- [27] PyTorch TorchAO Authors, “Torchao: Architecture optimization library for pytorch.” <https://github.com/pytorch/ao>, 2024. Libreria open-source per quantizzazione e compilazione ad alte prestazioni in PyTorch.
- [28] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [29] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Stanford alpaca: An instruction-following llama model.” https://github.com/tatsu-lab/stanford_alpaca, 2023. GitHub repository.
- [30] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [31] U. Kurt, “Which quantization should i use? a unified evaluation of llama.cpp quantization on llama-3.1-8b-instruct,” *arXiv preprint arXiv:2601.14277*, 2026.
- [32] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K. Zhang, and P. Luo, “Efficientqat: Efficient quantization-aware training for large language models,” in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [33] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, “Don’t stop pretraining: Adapt language models to domains and tasks,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 8342–8360, 2020.

- [34] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, “Carbon emissions and large neural network training,” *arXiv preprint arXiv:2104.10350*, 2021.
- [35] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.

Ringraziamenti

Ringrazio anzitutto il mio relatore, il docente Lorenzo Baraldi, per avermi fornito il materiale di base e i consigli necessari per sviluppare e indirizzare questa tesi con i relativi test. Inoltre, lo ringrazio per avermi concesso l'opportunità e le risorse per accedere all'infrastruttura di supercalcolo CINECA, consentendomi di condurre la sperimentazione empirica sul supercomputer Leonardo.

Ringrazio anche il prof. Francesco Guerra, per la sua enorme disponibilità nei confronti di tutti gli studenti e le studentesse durante questi anni accademici, sia come docente che presidente di corso di Ingegneria Informatica.

Ringrazio poi la mia famiglia, per avermi sostenuta in molteplici modi: ringrazio mia madre per avermi sempre detto che ero andata bene anche se non prendevo il massimo e la ringrazio per tutte le volte che mi ha accompagnata agli esami in modo che io potessi ripassare in auto; ringrazio mio padre per il sostegno economico e la disponibilità a venirmi a prendere in università o altrove quando non avevo l'auto, ma soprattutto perché ogni volta che chiedevo spazio o silenzio per studiare era il primo a lasciarmelo, anche se magari tornato da lavoro avrebbe voluto anche lui rilassarsi, guardando la televisione o chiacchierando in sala. Ringrazio mia sorella per il semplice fatto di essere mia sorella, di riempire la casa con la sua creatività e visione alternativa, che mi aiuta a riequilibrare il mio cinismo e la mia rigidità; la ringrazio per essere la persona con la quale posso non parlare mai e poi tirare fuori discorsi causali, sapendo che mi asseconderà. Sono consapevole di avere una famiglia con tante persone intelligenti, diverse e buone.

Ringrazio mia nonna Rosa, per esserci sempre stata; per essere sempre lì quando la cercavo, anche se magari io invece ero spesso altrove o poco disponibile. La ringrazio per tutti gli ottimi pasti che mi ha preparato, facendomi risparmiare il tempo di cucinare, che potevo usare per studiare o riposarmi. La ringrazio per tutte le preghiere che, come anche i miei genitori, ha rivolto per me. Ringrazio anche mio nonno Beppe, anche se non ha potuto vivere con me questi anni di università. Lo ringrazio per tutti i cruciverba fatti insieme e le

partite di scacchi; grazie a lui ho coltivato la mia curiosità e il mio interesse per la logica, ha assecondato gli stimoli mentali di cui avevo bisogno fin da bambina, che mi hanno portata fino all'attuale percorso di studio.

Ringrazio mia zia Manu con la sua costante propositività e solarità e la piccola Ylenia. Ringrazio mio zio Davide dal quale ho appreso tanto e che si è sempre interessato ai miei progressi nello studio; ringrazio mia zia Simona, sempre stata una ventata di freschezza, ringrazio Emily e Debora, la cugina con cui giocavo sin da bimba e con la quale avrò sempre un forte legame, anche se il crescere ci ha allontanate.

Ringrazio la mia bis-nonna Pierina, per il semplice fatto di essere qui, a vivere questa esperienza con me.

Ringrazio mia zia Laura, donna che stimo tanto e con cui ho vissuto un breve periodo di lavoro; ringrazio zia Teresa e zio Franco, persone alla mano e di grande intelligenza emotiva. Tre enormi esempi di altruismo.

Ringrazio mia nonna Pasqualina, che non si è mai dimenticata un compleanno e mi ha insegnato a fare i fusilli, mio nonno Luigi, sempre scherzoso; ringrazio mia zia Lorenza, la quale, ogni qual volta scendessi in Calabria, era pronta a offrirmi un posto per dormire o un pranzo.

Ringrazio Francesco, per essere un grande amico, oltre che cugino. L'unico con cui posso parlare letteralmente di tutto. Lo considero il tassello della famiglia che mi fa sentire compresa, sono grata della sua intelligenza a 360 gradi e della bella persona che è. Inoltre, è una continua fonte di scoperte, so che quando sono con lui sicuramente vivrò un'esperienza interessante.

Ringrazio tutti i miei famigliari.

Ora il grazie va invece ai miei compagni e amici: Livia, con me dalla prima superiore, abbiamo vissuto tutto questo percorso insieme e so che non è facile trovare un'amicizia con così tanti valori e interessi comuni; Sofia, una persona stupenda e una ragazza molto intelligente, scoperta con l'università, anche se abitava a solo qualche centinaio di metri da me. Non potevo aspirare ad avere amiche migliori per questo viaggio. Fabio, lui lo considero una meravigliosa riscoperta, un ragazzo che ammiro a livello didattico ma anche umano. Un grazie va anche ad Enrico, una di quelle persone che c'è senza fare rumore, ma quando non c'è lo noti. Voi avete reso questo percorso unico e bello, facendomi sentire comunque partecipe anche quando non c'ero. Mi avete stimolata a dare il meglio di me, perché siete i primi a dare il meglio.

Sono fortunata a poter condividere anche la magistrale con voi.

Grazie anche alla squad di Udu: ho veramente scoperto persone stupende, tra cui Elme il barqallino, Robbi (detta Bobbi), il grande Piace, Matri Pota, Tommy, Emanuele, Helena e Matteo.

In particolare però ci tengo a ringraziare Tau e Anna, voi siete quelli con cui ho stretto il legame più intimo. Con voi è tutto divertente, sapete scherzare e al contempo avere senso critico. Tau potrebbe parlarmi di tutto, che qualsiasi cosa dica risulta sempre interessante e articolata, persino quando scherza. Grazie per la tua unicità, Tau. Con Anna mi sento molto me stessa, magari non del tutto, ma molto; e io non mi sento quasi mai me, mi sembra di guardare il mondo da un oblò o vivere indossando il costume da giullare. Quindi grazie per la tua dolcezza e banalmente per la tua persona.

Un grazie speciale va anche a Roberta, in realtà, per avermi dato la possibilità di condividere con lei qualsiasi pensiero, anche poco convenzionale nel contesto in cui ci trovavamo insieme, e per non avermi mai giudicata.

No Elena, non ti ringrazio perché mi giudichi per usare Copilot; però sei stata una piacevole scoperta.

Grazie, inoltre, a chi mi ha accompagnata nel percorso scolastico: Jep e SabrinA, ormai non più nel team di studio, ma comunque indimenticabili.

Grazie anche al prof. Roberto Zanasi, che mi ha aiutata a coltivare i miei interessi in ambito scientifico e, in particolare, per la matematica. Continuiamo a portare in alto il buon nome del Fermi: l'eccellenza.

Grazie alla prof.ssa Natali, che è stata la prima a farmi conoscere quella che poi è diventata la mia lingua preferita: C.

Grazie a Elena Gavioli, docente di italiano e storia durante il triennio all'I.T.I.S. E. Fermi. Mi ha dato e insegnato tanto, non solo alle superiori: dopo il primo anno di università, infatti, mi disse che prendere voti alti quando se ne hanno le capacità, non significa arrendersi alle aspettative, ma dimostrare di aver acquisito conoscenze complete e durature, aver raggiunto il proprio potenziale ed essere esperte. La sua riflessione mi ha aiutata a non adagiarmi, cercando sempre buoni risultati.

*La gratitudine è l'espiazione
di inadeguatezza e immeritati meriti.*

Grazie.